VNUHCM - UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

CSC14005 - Introduction to Machine Learning

LAB 01

VIETNAM TRAVEL PLANNER

Class: 23KHDL1

Group 3:

Lê Quang Phúc - 23127102

Nguyễn Quang Đăng Khoa - 23127212

Đoàn Thành Phát - 23127241

Trần Tiến Cường - 23127332

Trầm Hữu Nhân - 23127442

Instructors:

Mr. Bùi Tiến Lên

Mr. Lê Nhựt Nam

Mr. Võ Nhật Tân

April 15th, 2026

Contents

List of Tables

List of Figures

ASSIGNMENT TABLE

GitHub: ML Lab1.

Timeline: ML Lab1 Timeline.

No.	Full name	ID	Task	Complete
1	Lê Quang Phúc	23127102	Problem definition Data collection Modeling for Traditional paradigm README	100%
2	Nguyễn Quang Đăng Khoa	23127212	Problem definition Data collection Modeling for Deep Learning paradigm Application	100%
3	Đoàn Thành Phát	23127241	Problem definition Data exploration Modeling for Hybrid paradigm Application	100%
4	Trần Tiến Cường	23127332	Problem definition Data exploration Modeling for Hybrid paradigm Report	100%
5	**Trầm Hữu Nhân**	**23127442**	Problem definition Data collection Modeling for Deep Learning paradigm Report	100%

Table 1: Task assignment and member evaluation chart

No.	Assignment	Complete
1	Introduction and problem definition	100%
2	Data collection and data preparation	100%
3	Model design and implementation	100%
4	Model evaluation and Ccomparison	100%
5	Analysis and discussion	100%
6	Application development	100%
7	Conclusions and future work	100%

Table 2: Assessing assignment completion

1 Introduction and problem definition

1.1 Context

Ho Chi Minh City is Vietnam's largest economic, cultural, and tourism center. It attracts millions of domestic and international tourists each year thanks to its diverse historical sites, rich culinary culture, modern entertainment areas, and vibrant lifestyle. As user needs become increasingly diverse, finding and choosing a suitable destination amidst the vast amount of information available online becomes difficult and time-consuming. Ho Chi Minh City is Vietnam's largest economic, cultural, and tourism center. It attracts millions of domestic and international tourists each year thanks to its diverse historical sites, rich culinary culture, modern entertainment areas, and vibrant lifestyle. As user needs become increasingly diverse, finding and choosing a suitable destination amidst the vast amount of information available online becomes difficult and time-consuming.

1.2 Goals

With the aim of addressing the aforementioned challenges and contributing to the development and promotion of Vietnamese tourism, this project aims to build an intelligent Recommendation System using a machine learning model combined with evaluative reasoning. By collecting, analyzing, and evaluating a large amount of review data (comments) from popular platforms, the system can "understand" user needs through input descriptions, thereby optimizing and personalizing recommendations to suggest the most suitable locations.

1.3 Problem definition

To build an effective content-based recommendation system for text data, the team decided to approach and reframe the problem as **Multi-class Text Classification**. This would then be evaluated through post-processing steps to generate the final recommendation list.

- The model's input is text data with natural language characteristics in Vietnamese. Specifically, this includes user queries or comments.
- The model's output is a predictive probability distribution, with each label representing a specific location.

2 Data collection

2.1 Overview

The data collection process is fully automated, interacting directly with a dynamic web interface. It involves setting up lists of desired locations, simulating user behavior to avoid being identified as a bot by websites, and storing all data for later stages.

2.1.1 Source

The team confirmed from the outset that location data would be categorized into three types: **attraction**, **dining**, and **hotel**. Therefore, data for each specific category would be collected separately on three different major platforms:

- **Google Maps:** A data source for attractions, landmarks, museums, and entertainment venues.
- **Booking.com:** A data source for accommodation services such as hotels, homestays, resorts.
- **Foody.vn:** A data source for local cuisine, restaurants, and eateries.

2.1.2 Licenses

Before proceeding with the data collection, the system used `urllib.robotparser` to check and read the `robots.txt` files of all three platforms. The collection process fully complied with the allow/disallow directives for automated crawlers, raw data owned by the platforms (Google, Booking, Foody), and user-generated content.

This dataset is collected for **academic and educational purposes** within the framework of the course and is not used for any commercial or competitive purposes. Personal information of reviewers (if any) is also not extracted to protect their privacy.

2.1.3 Scope

All data is limited to **Ho Chi Minh City, Vietnam**, and was collected on **March 19, 2026** (the date of Lab 1 for the Introduction to Machine Learning course). The data will be updated and reflect the latest operational status of the collected locations.

2.2 Implementation plan

Before proceeding with the detailed installation process, establishing a detailed plan is absolutely essential. To monitor and ensure system functionality, control errors, and achieve desired results, the following steps will be taken:

- **Step 1:** Preparation and configuration
  - Identify common characteristics for all platforms, including: **name, address, rating, number of comments, price, category, operating hours, comments, score of each comment, URL**.
  - Analyze the structure and API call flows of all three websites to select the appropriate approach strategy.
  - Set up an anonymous Selenium WebDriver environment.
- **Step 2:** Development
  - Create separate classes for each web page to ensure encapsulation.
  - Program mechanisms for error handling, timeouts, and fallback strategies when an element is not found.
- **Step 3:** Execution and saving checkpoints
  - Tracking the data collection process in a real-world environment.
  - Implement temporary batch or geographic-based storage to prevent data loss in case of system crashes or network outages.
- **Step 4:** Aggregation and cleaning
  - Combine separate data files into a single general file.
  - Preprocess the data for subsequent stages.

2.3 Detailed implementation

2.3.1 Google Maps

The Google Maps platform was chosen as the data source to collect information about tourist attractions, historical sites, museums, and entertainment venues in Ho Chi Minh City (**category = "attraction"**). Because Google Maps uses a dynamic web architecture and frequently updates its interface, we designed the `GoogleMapsScraper` object class using the `Selenium` library to accurately simulate the behavior of real users.

- **Step 1**: URL harvesting
  - Establish a list of specific keywords in both **Vietnamese** and **English**.
  - Based on this keyword list, automatically search based on the central coordinates of Ho Chi Minh City with a zoom level of `12z`, limiting the results to within the city limits.
  - The browser automatically identifies the list of results and performs a scrolling loop using JavaScript. The scrolling stops at the end of the list or when the **limit of 200 locations** for each keyword is reached. The goal is to obtain all the locations' URLs to prepare for detailed analysis.
- **Step 2:** Parsing
  - The system collects **names, addresses, average ratings**, and **the total number of comments**. The team has developed fallback mechanisms to address changes in Google's HTML tag structure.
  - The source code dynamically handles special states such as "Open 24/7" or "Temporarily Closed" for the **operation hours** attribute.
  - **Ticket price** information has no fixed structure. The system must search for text in different tabs, using applied Regex filters to identify currency numbers or automatically convert foreign currencies.
- **Step 3:** Comments scrapping
  - The system scans the site and automatically downloads all comments for each location.

  - The maximum number of comments is limited to **50 per location**, and they must have been posted within the **last 5 years**.
  - For each comment, not only is the text content saved, but the user's rating is also extracted.

- **Step 4:** Data security and storage mechanisms
  - To prevent Google from blocking IP addresses or requiring Captcha, `Selenium` is configured to remove automation flags, change parameters, and apply a random sleep mechanism between operations.
  - If a location has already appeared in a previous keyword, the system will skip it to save resources.
  - To minimize the risk of network disconnections or browser crashes, after every **10** successfully collected locations, data is temporarily overwritten in the checkpoint file.

2.3.2 Booking

Booking.com was chosen as the data source to collect information about accommodations such as hotels, apartments, homestays, and resorts in Ho Chi Minh City (**category = "hotel"**). Because Booking.com's interface contains many complex interactive elements, dynamic pagination, and advertising forms that obstruct the view, the team continued to use object classes based on the `Selenium` library to automate the browser and handle these interface layers dynamically.

- **Step 1:** UI handling
  - The query is automatically generated with the destination "Ho Chi Minh City". To ensure consistency in collecting and comparing room prices, the system sets fixed parameters for check-in/check-out dates and default number of guests.
  - The website frequently displays login request dialogs or promotional advertisements upon page load. Team has integrated functions to automatically prevent these interruptions during the data scraping process.

- **Step 2:** Pagination

  - The system scrolls slowly, ensuring that all images, room rates, and utility labels are fully loaded into the **HTML structure**.
  - `Selenium` will continuously load new results pages until the page is full or the specified number of templates is reached.

- **Step 3:** Feature extraction
  - Collect the exact name, type, address, and geographical distance from the city center.
  - Extract the average rating, quality rating, and total number of customer reviews.
  - Record the room rate per night, using Regex to standardize the integer format. Also, scan any included featured amenity tags.

- **Step 4:** Data security and storage mechanisms
  - To prevent the website's security system from detecting automated data scraping and blocking IP addresses, the system applies random pauses between page transitions. `User-Agent` parameter simulation is also used to mimic real users.
  - The source code is refined to catch exceptions, safely assigning null values to new accommodations with no ratings or those temporarily out of room.
  - The data is continuously updated and ultimately output to a complete `CSV` file in tabular format.

2.3.3 Foody

The Foody.vn platform was chosen as the data source to collect information about food establishments, restaurants, eateries, and cafes in Ho Chi Minh City (**category = "dining"**). Due to the densely distributed data by region and the system's frequent access blocking mechanisms, the team applied a technical strategy combining API packet interception and DOM scraping to optimize performance and reliability.

- **Step 1:** Geographic iteration
  - The system is set up to browse through a predefined list of **24 districts/wards** of Ho Chi Minh City. Data collection by geographical area ensures city-wide data coverage and makes it easy to pinpoint and reprocess data if a local error occurs in a specific district.

- **Step 2:** Hybrid scraping strategy
  - Instead of reading the HTML structure from the outset, the system prioritizes intercepting implicit **API packets** called during page load. Data returned from APIs typically has a clean, detailed JSON structure, and extraction speed is significantly faster.
  - If the API is not captured, the system automatically activates a fallback option: directly scraping data from the **static DOM**. The system sets up a retry loop with a maximum of **3 attempts**, with a **2.5 second delay** after each attempt, helping the system overcome network page load delays without overloading the server.
- **Step 3:** Data validation and extraction
  - For a given location, the algorithm checks the active status flag, ignoring and not including locations that are no longer active in the results list.
  - For valid locations, the system performs a series of extractions and consolidations of key data fields, including: operating hours, price range, and number of comments.
  - Details of the overall score and component scores are extracted. The system retrieves written comments and includes the specific rating score for each comment.
- **Step 4:** Data security and storage mechanisms
  - To mitigate the risk of network outages or IP address blocking, the system employs an immediate storage mechanism. As soon as all the data for a district is scraped, the system will save it.
  - After the process is successfully iterated through all 24 districts, all these fragmented data files will be merged into a single complete dataset.

3 Data preprocessing

3.1 Context

This is a dataset compiling information collected from platforms (**Google Maps, Booking, Foody**) focusing primarily on accommodation, restaurants, eateries, and tourist attractions in Ho Chi Minh City (**"attraction", "dining"** and **"hotel"**). For ease of use in later stages, the team will consolidate the data into a single file.

- Data shape (6050, 10) - rows represent for locations, column represent for features.

3.2 Features description

Feature	Description	Data type
`name`	Name of location	str
`address`	Address of location	str
`score`	Ratinf score	float64
`nums_comments`	Total of comments	float64
`price`	Service price	str
`category`	Category	str
`hours`	Opearation hours	str
`comments`	List of comments	str
`comment_scores`	List of comment scores	str
`url`	Path	str

Table 3: Features description

3.3 Info

	name	address	score	nums_comments	price	category	hours	comments	comment_scores	url
0	*BOM HOMES* VINHOMES CENTRAL PARK- LUXURY APAR...	208 Nguyễn Hữu Cảnh, Quận Bình Thạnh, TP. Hồ ...	0.5	3.0	1350000	hotel	00:00 - 23:59	['Quá tệ. Booking xác nhận. Chủ nhà hủy ko báo...	[0,5; 0,5; 0,5]	https://www.booking.com/hotel/vn/bom-homes-vin...
1	1 AM Home Sai Gon	109/14 Đường Nguyễn Thiện Thuật Toà nhà có tần...	4.5	8.0	770000	hotel	00:00 - 23:59	['Phòng đẹp ơi là đẹp, rộng rãi thoải mái, sán...	[5; 5; 5; 4; 4; 4; 5; 4]	https://www.booking.com/hotel/vn/1-am-homestay...
2	1 Bedroom Apartment Cozy Canal View Hoang Sa s...	287 Đường Hoàng Sa, Quận 1, 123080 TP. Hồ Chí...	0.0	0.0	1399000	hotel	00:00 - 23:59	[]	[]	https://www.booking.com/hotel/vn/1-bedroom-apa...
3	1 Bedroom Apartment Landmark Plus Lp	208 Đường Nguyễn Hữu Cảnh, Quận Bình Thạnh, TP...	4.5	1.0	1288888	hotel	00:00 - 23:59	['The support and friendliness from the staff....	[4,5]	https://www.booking.com/hotel/vn/1-bedroom-apa...
4	1 Bedroom Daisy - Luxury Gold Apartment - Cent...	538/2/5 Đoàn Văn Bơ, 72800 TP. Hồ Chí Minh, ...	0.0	0.0	750000	hotel	00:00 - 23:59	[]	[]	https://www.booking.com/hotel/vn/1-bedroom-dai...

Figure 1: Data preview

```
<class 'pandas.DataFrame'>
RangeIndex: 6050 entries, 0 to 6049
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   name            6050 non-null   str
 1   address         6050 non-null   str
 2   score           5978 non-null   float64
 3   nums_comments   5978 non-null   float64
 4   price           4219 non-null   str
 5   category        6050 non-null   str
 6   hours           5296 non-null   str
 7   comments        6047 non-null   str
 8   comment_scores  6045 non-null   str
 9   url             6045 non-null   str
dtypes: float64(2), str(8)
memory usage: 472.8 KB
```

Figure 2: Metadata summary

3.4 Data standardization

This step aims to standardize the values in each column, eliminate noise, and convert the data types to suit subsequent analysis steps.

Column `name`

- Convert to string format, removing extra spaces and special characters.
- Standardize Unicode to NFC format to unify the representation of Vietnamese characters.

Column `address`

- Remove rows with empty or null addresses.
- Remove spaces, redundant characters, and normalize Unicode NFC data similar to the ‘Name‘ column.
- Remove country suffixes, city name variations, and postal codes.

Column `score`

- Convert to floating-point type, replace invalid values with 0, and cast to `float32` to save memory.

Column `nums_comments`

- Convert to integer type, replace invalid values with 0, cast to `int32`.

Column `price`

- Split into two columns: `price_min` and `price_max`.
- Missing or invalid values are assigned 0.
- Delete the original Price column after splitting.

Column `category`

- Remove spaces, capitalize the first letter, and switch to the `category` style to optimize memory usage and support grouping/filtering operations.

Column `hours`

- Separate into two columns `start` and `end`.
- If a value is missing or incorrectly formatted, set the default to "-".
- Delete the original Hours column after separation.

Column `comments`

- Standardize the comment list to a consistent format `[{comment 1}, {comment 2}, ...]`.
- Remove newline characters (\n), reduce whitespace, and delete extra characters.
- Empty values are assigned to the empty list "[]".

Column `comment_scores`

- Converts a string delimited by semicolons ";" into a Python `list float`.
- Empty values are assigned to the empty list "[]".

Column `url`

- Shorten URLs by removing the query string and the `/data=...` part containing unnecessary metadata.

3.5 Remove duplicates

Duplicate data will be defined by the team as locations with the **same** name and address. The team will delete these and keep only one unique location.

3.6 Check data valid

Perform data validation by checking if the values of **numerical features** are valid, for example, `score` is within the range [0, 5], and values such as `nums_comments` and `price` are greater than 0.

3.7 Column `address`

After the normalization step, the `Address` column had its country suffix, city name, and postal code removed. However, the addresses were still in raw form with various formats (missing ward/district, containing building names, inconsistent order, etc.). This step involved calling the **Gemini API** to normalize all addresses to a unified format: `"[house number] [street name], [ward], [district]"`, in order to provide complete location information and facilitate the extraction of wards/districts in subsequent analysis tasks.

Detailed implementation:

- **Unique address extraction:** Retrieves a set of non-duplicate `Address` values to minimize **API calls** (instead of processing each row, only processes distinct addresses).
- **Batching:** Groups addresses into batches of **50 addresses/request** due to the limitations of the free API key.
- **Gemini API call:** Each batch is sent with a prompt requesting LLM to normalize the address according to the following rules:
  - Keep the house number and street name.
  - Infer the ward/district if missing but identifiable from the context.
  - Remove auxiliary information (building name, description, floor number, etc.).
  - Return results in `JSON` format with a fixed schema (`id`, `address`) with `temperature=0.1` to ensure consistency.
- **Error handling:** Each batch is tried a maximum of **3 times**. If it still fails, the original address is kept to avoid data loss.
- **Reverse mapping:** Build a mapping table from raw addresses to normalized addresses, then update the entire `Address` column in the original DataFrame.

Results: All addresses in the `Address` column are converted to a standard format, unifying the data structure and facilitating the extraction of district and ward information for subsequent analysis and visualization steps.

4 Data visualization

4.1 Univariate analysis

The goal of this section is to examine the distribution characteristics of each variable independently, including assessing the distribution shape, identifying outliers, and recognizing notable locations for each individual indicator.

4.1.1 Category distribution (`category`)

The goal of this section is to identify the component structure of the dataset according to three main categories using two charts:

- Using **pie chart** to show the percentage of each category in the entire dataset, helping to identify which category is dominant.
- Using **bar chart** to show the absolute number of locations in each category, allowing for direct comparison of scale between groups.

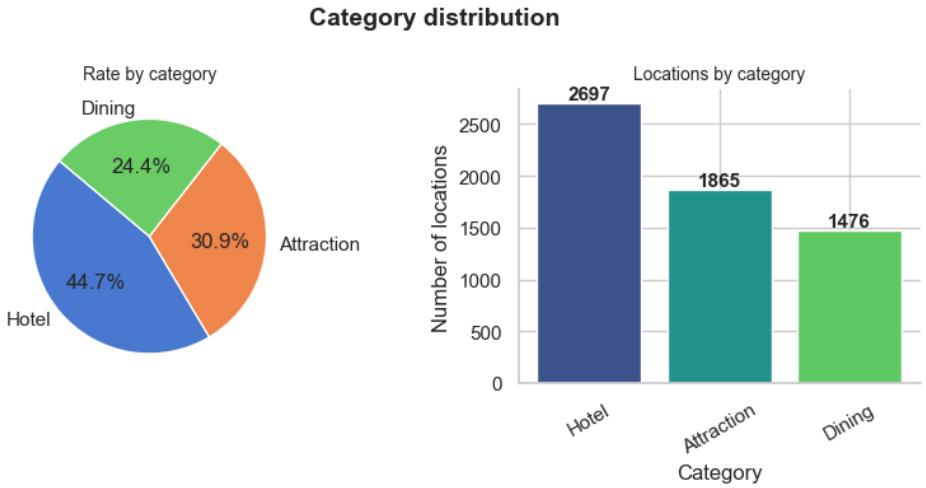

Figure 3: Category distribution

Insights:

- Dataset focuses primarily on the `hotel` category, accounting for an overwhelming **44.6%** (equivalent to 2698 locations).

- The `attraction` and `dining` categories account for lower percentages, at **30.9%** (1870 locations) and **24.4%** (1478 locations) respectively.
- This disparity indicates a slight imbalance in the dataset, favoring accommodation services over entertainment and dining venues.

4.1.2 Distribution of scores, prices, and comments

The goal of this section is to analyze the distribution shape of continuous variables in the dataset through four **histograms** combined with **density curves (KDEs)** plotted for each variable:

- `Score` distribution of the average rating of locations.
- `Price_min / Price_max` distribution of service price ranges, often **right-skewed** due to the presence of high-end locations.
- `Nums_comments` distribution of the number of comments, reflecting the popularity and engagement of each location.

Statistical indicators skewness, kurtosis, and median are displayed directly on each histogram to aid in quick evaluation.

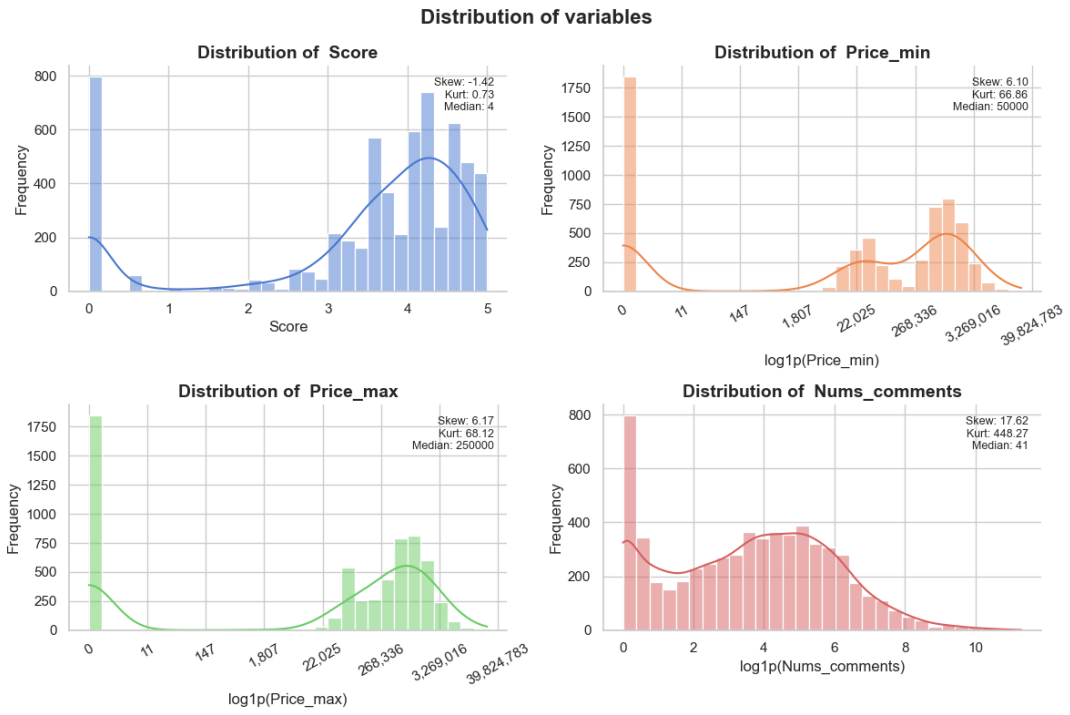

Figure 4: Distribution of variables

Insights:

- `Score` variable is **left-skewed**, mainly concentrated at high scores (median is 4), but a large amount of data is clustered at 0 points (locations that have not received reviews).
- The price variables (`price_min, price_max`) and interaction volume (`nums_comments`) are both extremely **right-skewed** with very high kurtosis.
- This indicates that most locations are in the budget price segment and have few comments (median is only 42), but there is a small group of outliers with extremely high values that need to be considered before modeling.

4.1.3 Outliers by variable

The goal of this section is to identify the core distribution range and detect outliers of continuous variables (`score, price_mean, nums_comments`) through four boxplots for comparison:

- Box (IQR) covers **50%** of the central data (from Q1 to Q3).
- Whistle represents the range of valid data according to the **1.5×IQR** rule.
- Red dots observations that fall outside the IQR threshold, identified as **outliers**, the number and percentage of outliers are noted below each chart.

Figure 5: Distribution and outliers

Insights:

- All four variables recorded a significant percentage of outliers, ranging from **6.3%** to **14.5%** of the total observations.
- For the `score` variable, outliers were concentrated at very low scores (below 1.5), reflecting a small group of poorly rated or unrated locations.
- Conversely, the price variables (‘`price_min, price_max`‘) and interaction variables (`nums_comments`‘) showed persistent outliers at their maximum values, indicating a large disparity between the group of ordinary budget locations and a small number of ultra-luxury or highly viral locations.

4.1.4 The highest rating with the most comments

The goal of this section is to rank the locations with the highest service quality based on user reviews. To ensure statistical significance, only locations with **over 2,000 reviews** are included in the ranking — excluding locations with fewer reviews that may have high scores but lack representativeness. The ranking is represented by a horizontal bar chart showing the **Top 10 locations** ranked in descending order of score, with direct value labels on each bar.

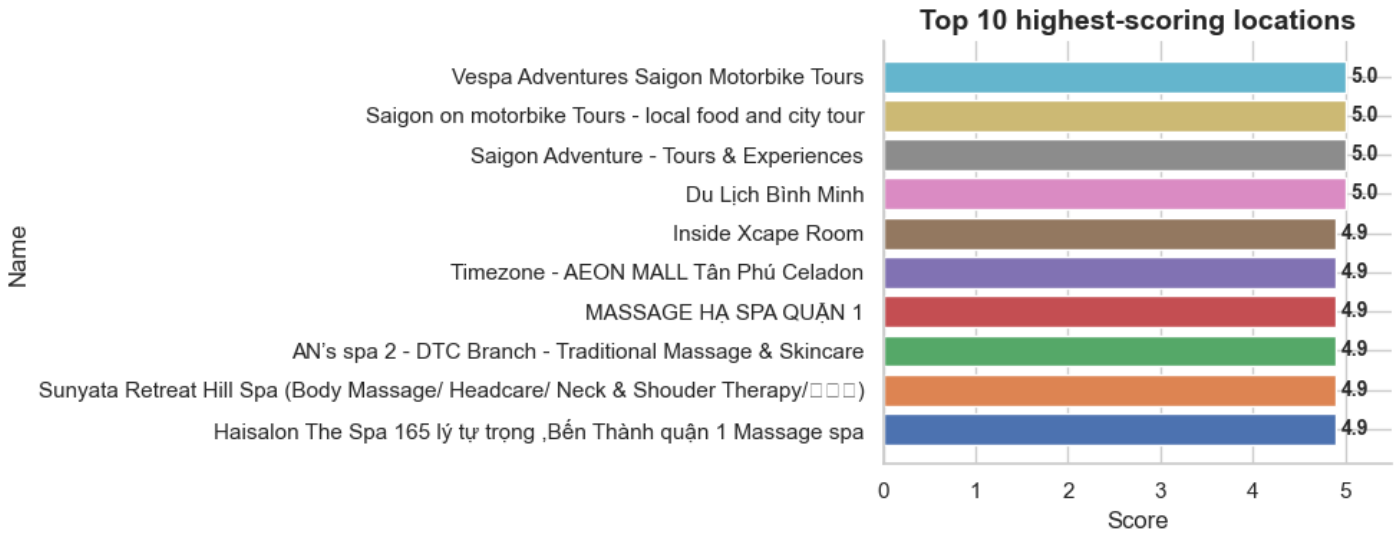

Figure 6: Top10 highest-scoring locations

Insights:

- The top 10 highest-scoring locations received near-perfect ratings, ranging from **4.9** to **5.0**.
- The leading group in the rankings mainly consists of services offering unique experiences such as motorbike tours, street food tours, and wellness facilities (spas, massages).
- This reflects the superior level of user satisfaction with business models that provide deeply personalized experiences, rather than mass-market accommodation or dining services.

4.1.5 Highest number of comments

The goal of this section is to identify locations with the highest level of community engagement through the number of comments on a horizontal bar chart. The top 10 locations are ranked in descending order of comment count; corresponding scores are displayed to compare popularity and perceived quality.

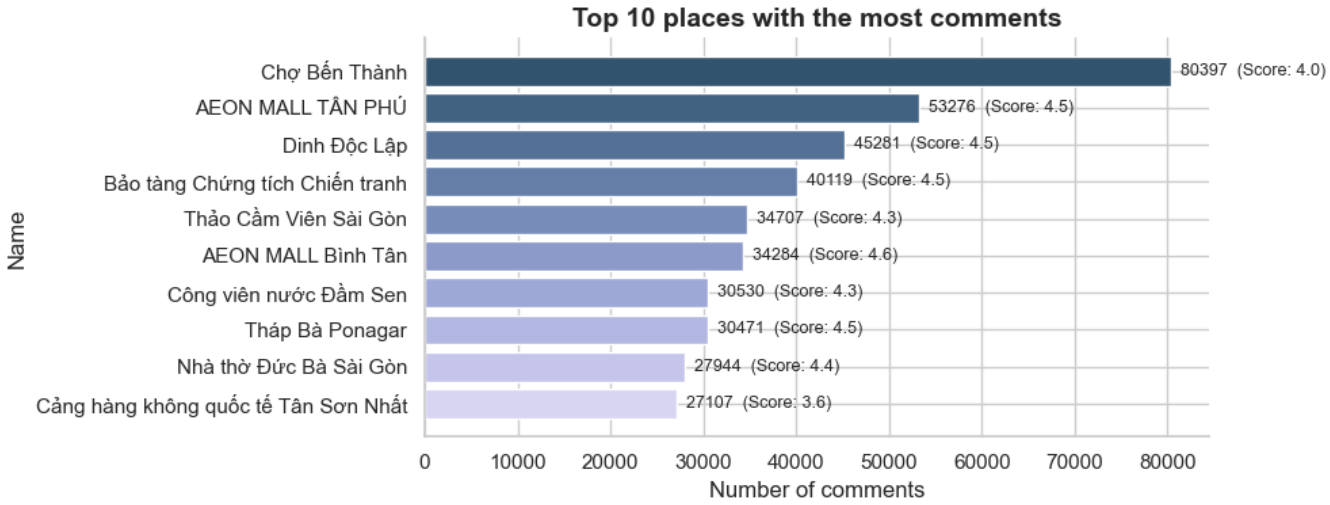

Figure 7: Top10 locations with the most comments

Insights:

- **Ben Thanh Market** attracted the most interaction with over 80,000 comments, creating a significant gap between it and the second place location.
- The top 10 most popular locations mainly focused on iconic **cultural** and **historical** landmarks and large-scale shopping malls.

- Notably, the massive volume of reviews did not entirely reflect the level of satisfaction; while commercial areas maintained very positive scores (4.5 - 4.6), public areas such as airports or traditional markets recorded more modest scores (3.6 - 4.0).

4.2 Multivariate analysis

The goal of this section is to explore the **relationships** and **interactions** between variables, including category-based comparisons, correlation analysis, geospatial visualization, and analysis of combined indicators such as Value-for-Money.

4.2.1 Comparison by category

The goal of this section is to synthesize and compare the overall performance of each location category across three key **metrics** using three grouped bar charts for illustration:

- **Average of score** compares the average service quality across the `attraction, hotel`, and `dining` categories.
- **Average of price max** reflects the typical price segment of each category.
- **Average of comments** measures the level of engagement and relative popularity of each category.

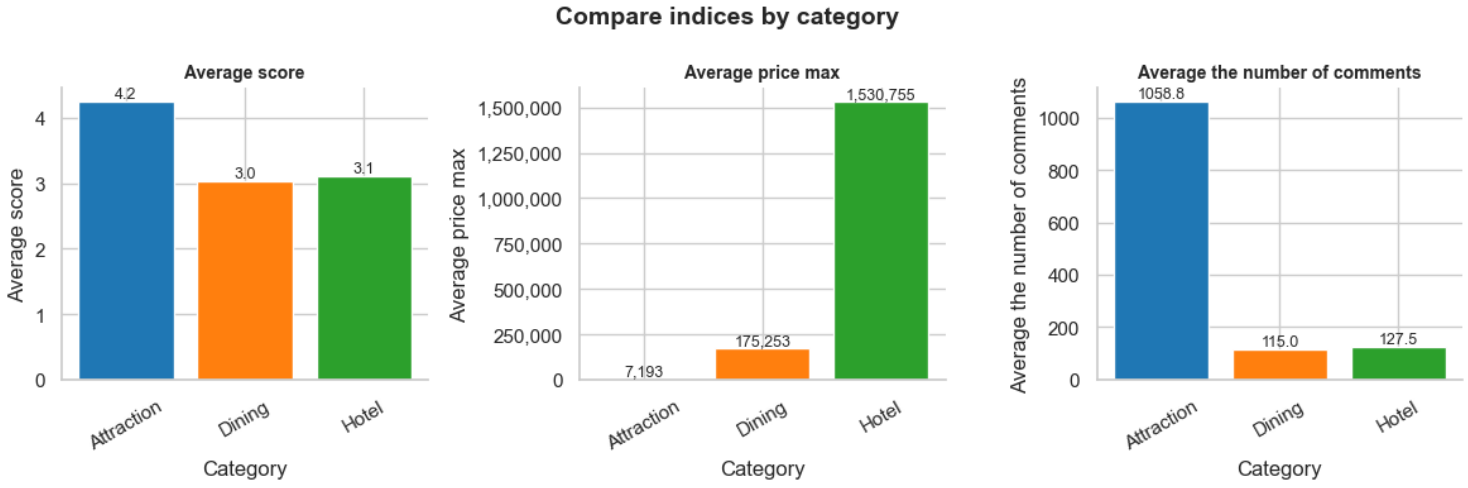

Figure 8: Compare indices by category

Insights:

- The `attraction` group recorded the highest level of interaction and satisfaction, with an average score of **4.2** and nearly **10 times** more comments (1061.2) than the other groups.
- Conversely, the `hotel` group had the biggest financial barrier with a significantly higher average maximum price (over **1.5 million VND**), far surpassing the dinnings and attractions.
- Notably, despite the huge price difference, both the `hotel` and `dining` groups shared relatively modest average scores (**3.0 - 3.1**) and low interaction levels, indicating that service costs are not directly proportional to user satisfaction.

4.2.2 The relationship between scores and the number of comments

The goal of this section is to examine the relationship between the **number of comments** and**score** of locations.

- Category scatter plot point represents a location, colors distinguish categories.
- Log line trend reflects the overall trend between two variables, where the X-axis (`nums_comments`) is represented on a logarithmic scale to handle **heavily skewed distributions**.
- Pearson Correlation Coefficient (r) is directly displayed on the chart header to provide a quantitative measure of the degree of linear correlation.

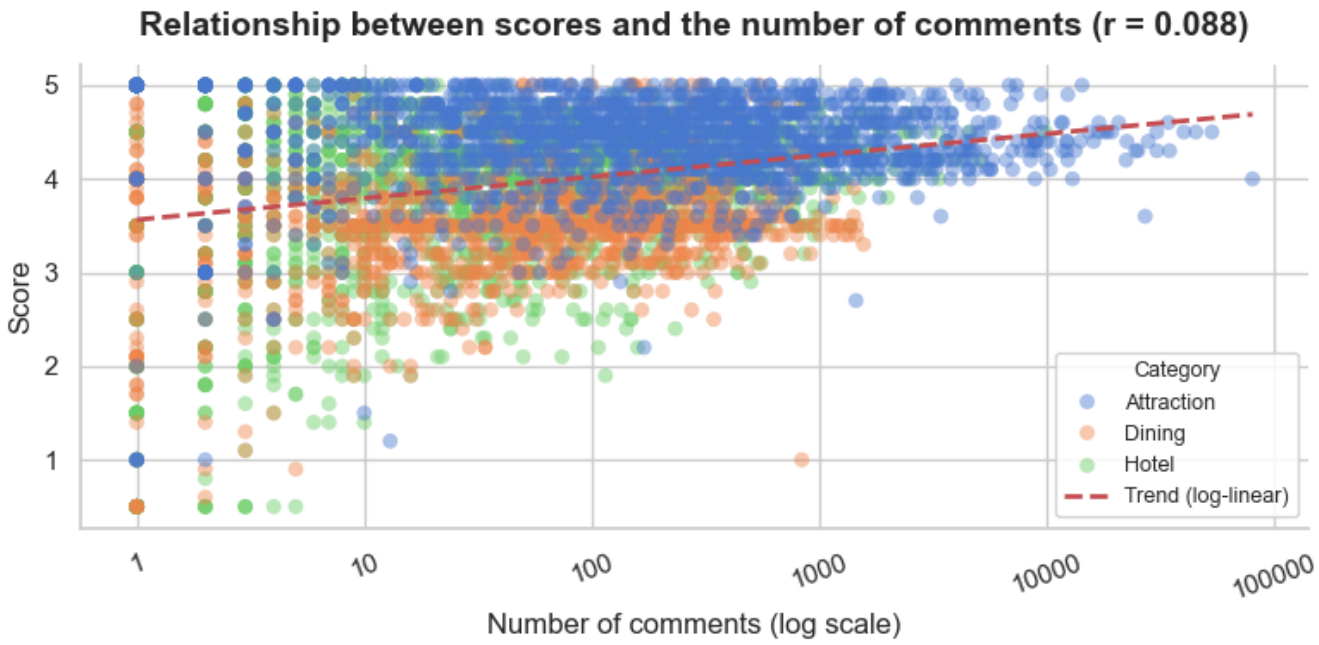

Figure 9: Scores vs. the number of comments

Insights:

- The low correlation coefficient (**r = 0.089**) indicates that there is almost no clear linear relationship between the number of reviews and the rating of a location.
- Attractions completely dominates the high-scoring and high-engagement areas (exceeding **10000 reviews**), suggesting that popular tourist destinations generally maintain very high levels of satisfaction from the public.
- Meanwhile, Hotel and dining groups are more scattered across the lower-scoring ranges and rarely attract more than **1000** reviews, reflecting the fragmented nature, conflicting opinions, and difficulty in satisfying the majority of these two service groups.

4.2.3 Value for money by category

The goal of this section is to assess the **"value for money"** of each location category based on the correlation between quality and cost.

- The `Value for money` index is defined as: Value index = $\frac{\text{Median Score}}{\log_{10}(\text{Median Price})}$

- Horizontal bar chart compares the value for money index between categories — a higher value indicates better quality for the price paid.
- The corresponding median price is indicated with a label for each column for easy reference.

Note:

- Value for money by category (Index = Median score / $\log_{10}$(Median price))
- Value for money index (a higher value indicates better value for money)

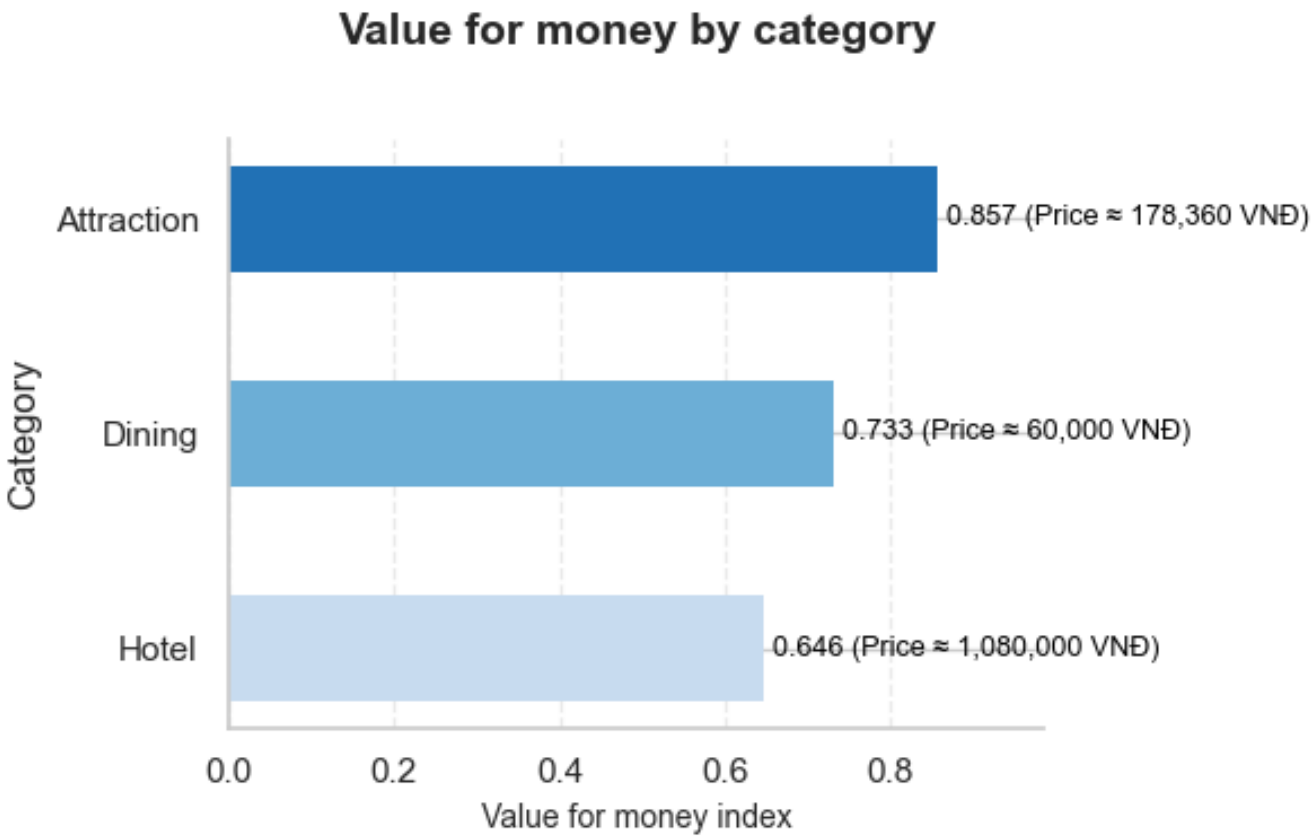

Figure 10: Value for money by category

Insights:

- Attractions lead in terms of `value for money` (index of **0.857**) with a mid range cost (approximately **178,000 VND**), indicating that users highly value the experience received compared to the amount spent.

- Conversely, the hotels, despite having the highest median price (over **1 million VND**), recorded the lowest index (**0.646**), reflecting the fact that satisfaction does not increase proportionally with the high cost of accommodation.
- Dinings offer a fairly good value for money (**0.733**), mainly due to the advantage of very low access costs (approximately **60,000 VND**), suitable for the vast majority of customers.

4.2.4 Distribution of scores by category

The goal of this section is to compare the distribution of scores across location categories, while also identifying differences in median and dispersion.

- Boxplot displays the interquartile range (IQR), median, and outlier values of the `score` for each category.
- Strip plot displays individual data points (with high transparency) to supplement the picture of the actual density distribution of the underlying data.

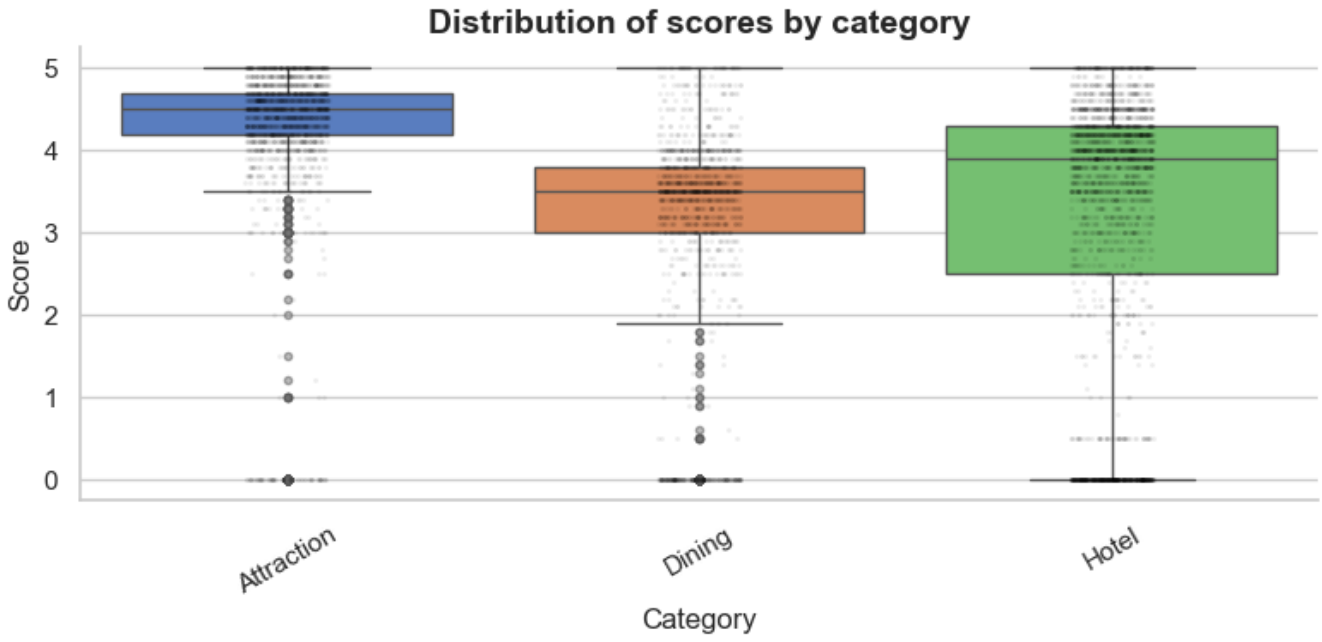

Figure 11: Distribution of scores by category

Insights:

- Attractions recorded the most stable and outstanding service quality with a high median score (approximately **4.5**) and a narrow data distribution range (IQR), mainly concentrated in the excellent score range.
- In contrast, Hotels group showed strong differentiation in quality with a very wide range of scores, spanning from low average (**2.5**) to high (**4.3**).
- Dinings had the lowest overall average (median around **3.5**). At the same time, all three categories showed many outliers extending down to the 0 mark, most likely locations that have not received actual evaluations.

4.2.5 Distribution of prices by category

The goal of this section is to compare the service price segments across different location categories. Because the price variable has a strongly **right-skewed** distribution with a very wide range of values, the chart uses a **logaric scale** on the Y-axis to ensure readability and comparability.

- Boxplot shows the average price distribution range (`price_mean`) of each category.
- Strip plot shows the actual dispersion of price data within each group.

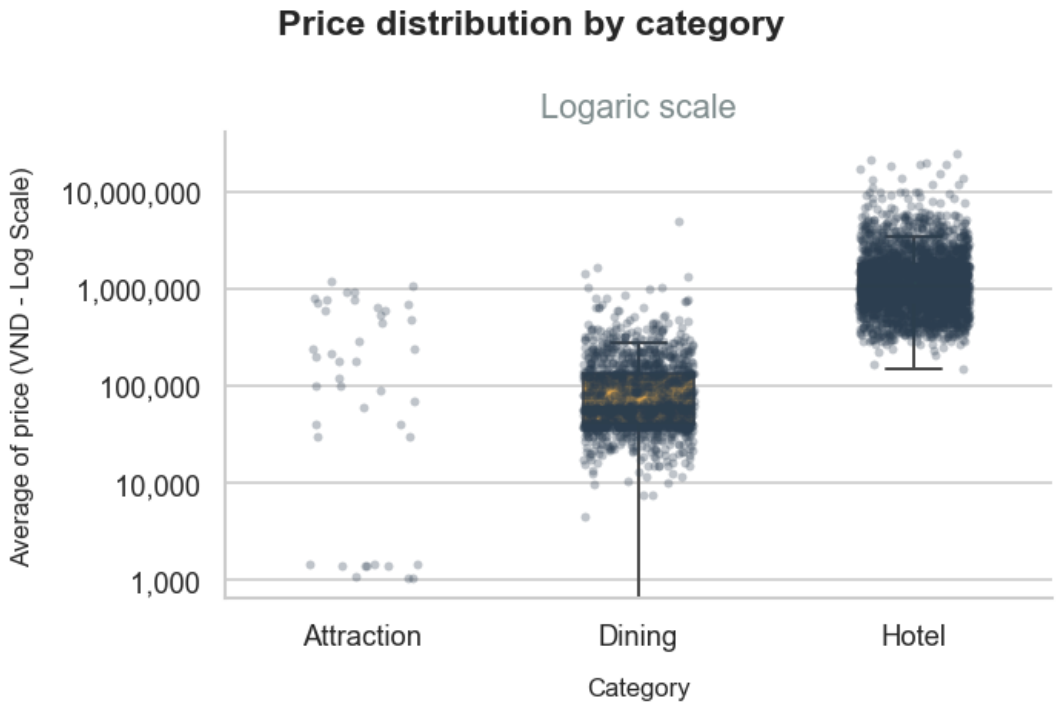

Figure 12: Distribution of prices by category

Insights:

- There is a clear differentiation in cost levels between services the `hotel` category has a high price range (median around **1 million VND**), while the `dining` category mainly serves the budget segment (ranging from **40000** to **150000 VND**).
- Both the hotels and dinings have a wide distribution range with many outliers reaching very high prices, indicating a diverse market offering everything from budget-friendly options to luxury services.
- The attractions, in particular, have very sparse and highly dispersed price data, reflecting the unique reality of this group where many public places are free to enter.

4.2.6 Correlation between variables

The goal of this section is to visualize the degree and magnitude of linear correlation between the principal variables in the dataset.

- Pearson correlation matrix is calculated and displayed as a heatmap for four variables: `score`, `nums_comments`, `price_min`, and `price_max`.
- A diverging colormap is used to clearly distinguish between positive correlation, negative correlation, and uncorrelated areas; the correlation coefficient (`r`) is noted directly in each cell.

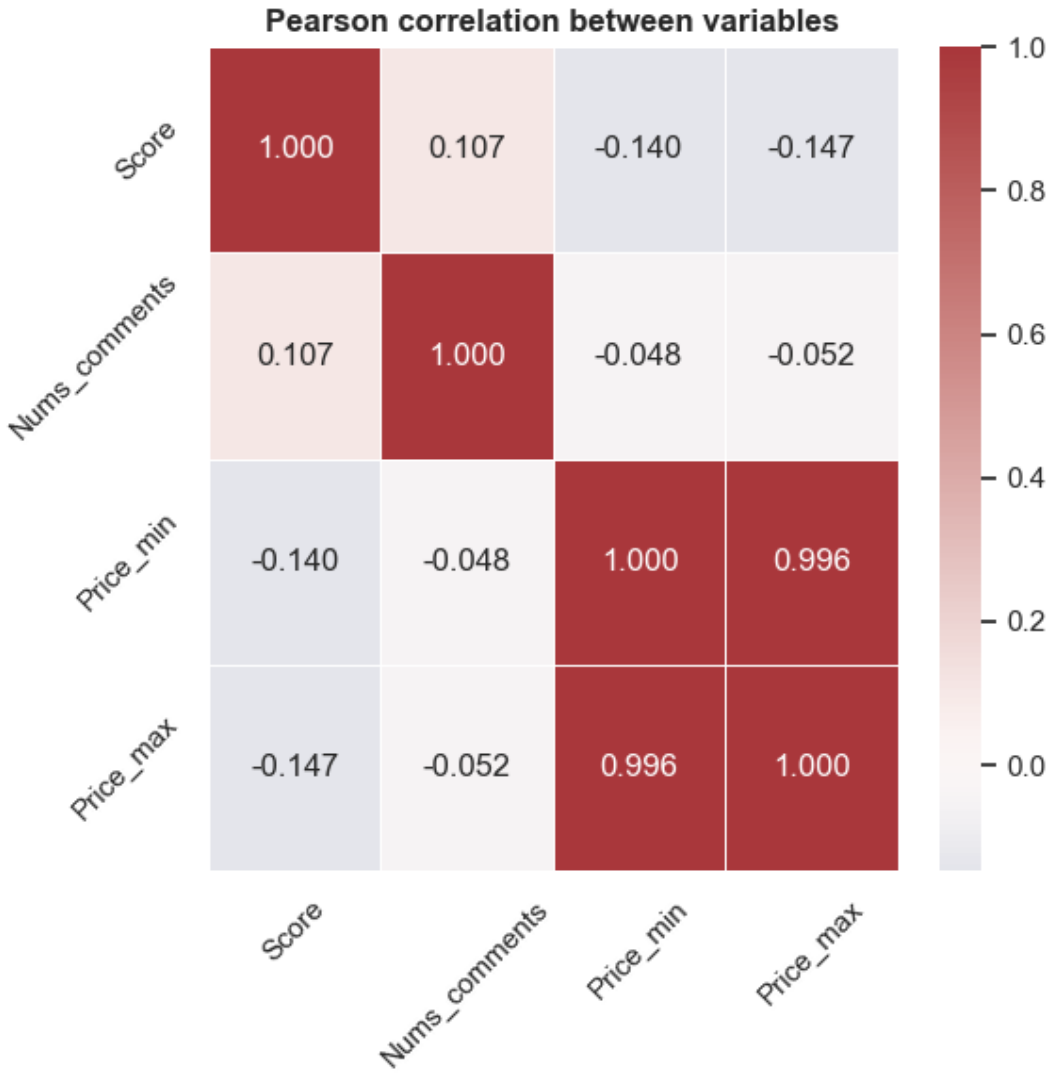

Figure 13: Pearson correlation between variables

Insights:

- The two variables `price_min` and `price_max` have a nearly absolute positive linear correlation ($\mathbf{r = 0.996}$), reflecting very strong multicollinearity, suggesting that they should be combined into a single average value variable to optimize the machine learning model later.
- `Score` and `nums_comments` are almost independent of each other ($\mathbf{r = 0.107}$), demonstrating that a location with high coverage and many comments does not necessarily mean it will achieve an excellent quality score.
- At the same time, the price shows a very weak negative correlation with the rating ($\mathbf{r =}$

-0.14), implying that customers tend to be a little more critical and more likely to give lower scores when experiencing services with higher costs.

4.2.7 Distribution of operating hours

The objective of this section is to analyze the operating time patterns of locations based on opening and closing time data.

- Opening time bar chart shows the frequency of opening hours.
- Closing time bar chart shows the frequency of closing hours.
- Opening - Closing heatmap represents the combined distribution of two time variables to identify common operating hours.
- Locations operating 24/7 are excluded from the opening/closing time distribution analysis to clarify the trends of the remaining hours.

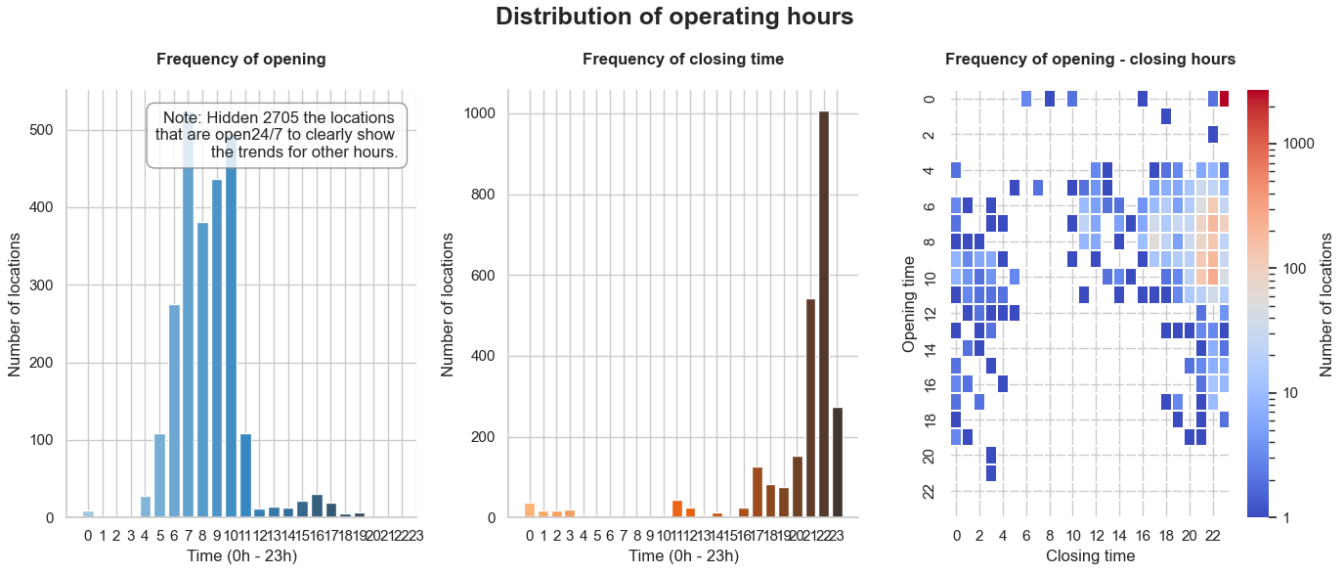

Figure 14: Distribution of operating hours

Insights:

- The most striking feature of the data is the prevalence of 24/7 operating hours, with over **2700 locations**, indicating a very high level of availability to serve customers around the clock.

- For businesses with fixed operating hours, the majority choose to open in the morning (heavily concentrated from **6 am** to **10 am**) and close late in the evening (**9 pm** to **11 pm**, peaking at **10 pm**).
- The heatmap reinforces this trend with clusters of warm colors converging at corresponding time pairs, reflecting the commercial pace and service demand extending from early morning to late night.

4.2.8 Normalize addresses to coordinates

The goal of this section is to assign geographic coordinates (`latitude, longitude`) to each location in the dataset to facilitate spatial data visualization on a map.

- The `extract_address_info` function separates the `address` column into **ward, district**, and **street**, and normalizes district names through the `district_alias` table.
- Geocoding using Nominatim API, unique addresses are sent to the **Nominatim (OpenStreetMap)** service to obtain `(lat, lon)` coordinates with a limit of **1 request/second** according to the API policy.
- For addresses whose coordinates cannot be found, the system assigns district/county center coordinates from the `hcm_district_coords` table to minimize data loss.
- The final result is stored in two new columns of the DataFrame: `lat` and `lon`.

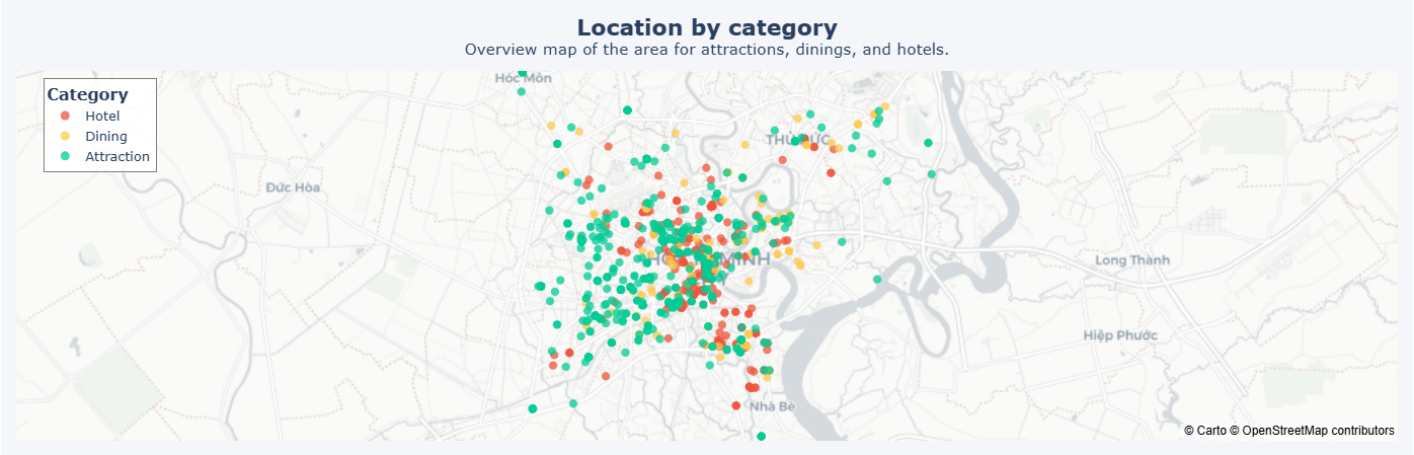

Figure 15: Location by category

Insights:

- The highest concentration of service facilities is found in the city center.
- In contrast to the clustering of accommodation, the attractions and dinings show a more widespread distribution and expansion into neighboring districts and counties.
- The close intermingling of these three categories on the map clearly reflects the formation of integrated tourism-service ecosystems, comprehensively meeting the needs of tourists in the same area.

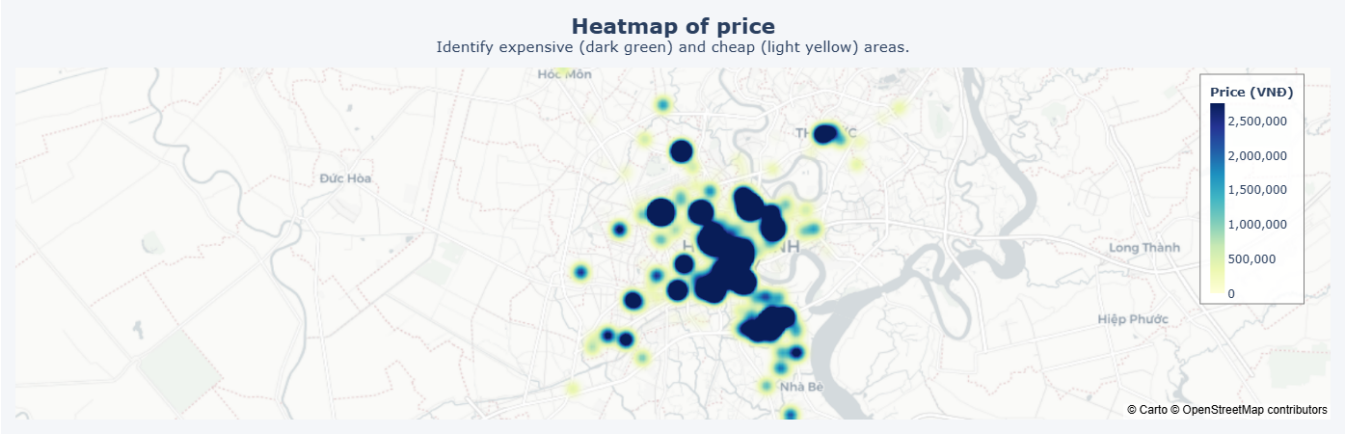

Figure 16: Heatmap of price

Insights:

- The heat map shows a clear spatial differentiation in cost, with the most expensive services (dark green area, prices above **2 million VND**) densely concentrated in large clusters in the city center.
- Conversely, when expanding to the suburban districts and areas, the color intensity decreases significantly and shifts to a light yellow tone, indicating the dominance of the affordable service segment (below **500000 VND**).
- This pricing positioning closely reflects the law of commercial land rent, confirming that service costs are directly proportional to the prime location and centrality of the geographical position.

4.2.9 Location density and pricing by area

The goal of this section is to visualize the geographical spatial distribution of locations on a map of Ho Chi Minh City, combined with an analysis of price differences across different areas.

- Quantity density chart analys the **top 15** districts with the most locations, color-coded by category.
- Average price bar chart (`hotel`) compares average accommodation prices across districts.
- Average price bar chart (`dining`) compares average dining costs across different areas.
- Average price bar chart (`attraction`) compares sightseeing costs across different areas.

The charts share the same Y-axis (district name) for convenient comparison of quantity and price levels across each geographical area.

Figure 17: Quantity and price by category in districts

Insights:

- In terms of density, **district 1** serves as the core accommodation and tourism hub, with an overwhelming number of hotel establishments compared to the rest of the city.
- However, in terms of cost, **Thu Duc city** surprisingly has the highest average hotel room price (exceeding **3.5 million VND**), creating a significant gap compared to the city center.
- In other categories, the price segment also shows an interesting differentiation, **district 4** leads in average food costs, while sightseeing costs remain extremely low throughout the district, only slightly higher in **districts 5** and **1**.

4.3 Text analysis

The goal of this section is to extract information from **unstructured data**, including text-based comments and location names, to add a qualitative perspective to the overall analysis.

4.3.1 Keywords featured in comments and location names

The goal of this section is to visualize the most frequently appearing keywords in comments and location names.

- Word cloud (`comments`) aggregates all comment text from the `comments` column, highlighting characteristic words frequently mentioned by users (e.g., service quality, location, experience, etc.). Common Vietnamese and English stopwords have been excluded to avoid noise.
- Word cloud (`name`) aggregates the names of location in the dataset, helping to identify common keywords in the names (e.g., district name, type of service, etc.).

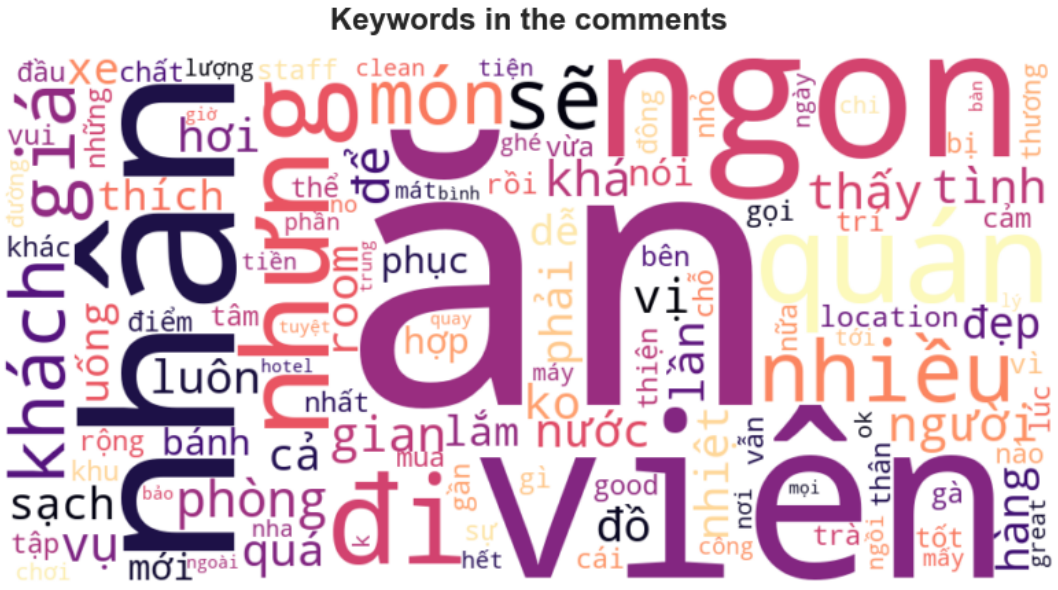

Figure 18: Keywords in the comments

Insights:

- The most prominent keywords are “ngon”, “quán” and “nhân viên” indicating that customer reviews focus primarily on food quality and service attitude.
- The significant appearance of words like “nhưng" and “không" suggests that many comments include comparisons or two-sided reviews (praise and criticism).
- Words related to service experience such as “phòng”, “sạch sẽ", “vị trí” and “giá" also appear, reflecting customer interest in space and amenities beyond food.

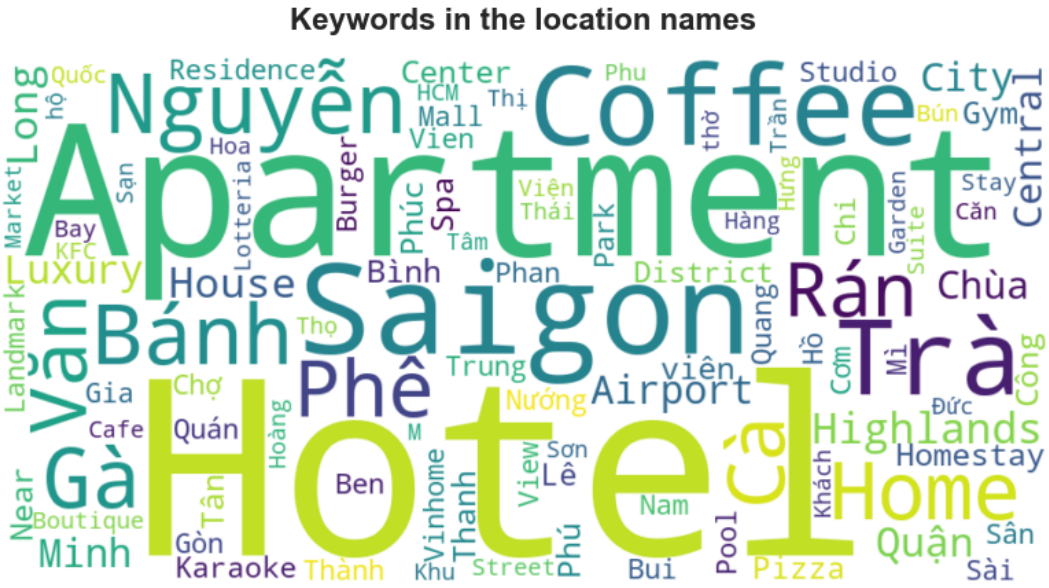

Figure 19: Keywords in the location names

Insights:

- Nouns identifying service types such as “Hotel", “Apartment",“Coffee" and “cà phê" dominate, reflecting the overwhelming number of accommodation and beverage establishments in the dataset.
- Geographical location factors are thoroughly exploited in brand naming, as evidenced by the frequent appearance of the keyword “Saigon" and popular street names (such as “Nguyễn”).
- For the food group, highly distinctive and easily recognizable dishes such as “Gà", “Bánh" or “Trà" are often directly included in the names by business owners to attract target customers.

5 Modeling

5.1 Transform the problem

The team's goal is to build a **recommendation system** that suggests the most relevant tourist destinations to users in a specific context. In practice, a tourist destination recommendation system typically receives user descriptions as input; directly generating a list of destinations from raw text is **a complex task** and **difficult to control quality**. Therefore, the team decided to frame this problem as a **multi-class classification** problem within the core machine learning architecture.

5.1.1 Post precessing and inference

The raw output from the machine learning model is not the final recommendation. The team performs the following post-processing steps:

- The multi-class classification model will return a probability distribution corresponding to each location.
- Locations will be sorted in order of predicted probability from highest to lowest.
- Filter the list of locations based on the information provided by the user.
- Establish a list of suggested locations (the final result displayed to the user) including **3 attractions, 2 dinings**, and **1 hotel**.

5.2 Machine Learning problem

After transformation, the core machine learning problem is rigorously defined as **supervised learning – multi class text classification**.

- **Input:** A set of texts containing comments from locations (The team will treat each comment as a description from a real user.).
- **Output:** Location names (labels) corresponding to the input comments (each location will be a different classification).

- **Objective:** Find a mapping function f such that the model can correctly assign labels to a new, unprecedented query, minimizing the difference between the predicted probability distribution and the actual labels.

5.3 Pipeline

- Preprocessing and features engineering
- Data splitting
- Model training
- Model evaluation and final retraining
- Post processing and system inference

5.4 Basis for comparison

To ensure that the comparison between the three paradigms is scientific, objective, and fair, the group established a rigorous common frame of reference.

5.4.1 Same data foundation

- The models are required to use a common format for processed input data (`processed_data.csv`).
- The Train/Valid/Test split strategy is exactly the same, with a common ratio of **70/15/15**. Regardless of the model, the observation set used for training and the set used for testing remain absolutely unchanged. This is achieved by fixing `random_state` so that the results of each data split will be the same.

5.4.2 Evaluation metrics

The output of each model is standardized to be measured by two metrics to comprehensively evaluate the multi-class classification problem:

- **Top-5 Accuracy** measures the accuracy rate of predicting whether the location of the input comment falls within the top 5 locations predicted by the model with the highest probability. This reflects the accuracy of the model.

- **Macro F1-Score** calculates the F1 for each location (class) separately, then averages the classes regardless of their numerical weights. This ensures the model provides good recommendations for all locations, avoiding the misleading effect of repeatedly suggesting only the most popular locations.

5.5 Deep learning: TextCNN

The system uses **Deep learning** paradigm as its core to solve the problem of problem solving, applying tips based on content (recommendations) and restating them in the form of a multi-class text classification problem.

5.5.1 Model architecture

Group improved upon the **TextCNN** (Convolutional Neural Networks for Test) structure proposed by Yoon Kim, 2014 incorporating pre-trained Word Embedding with the following key components:

- **Input Layer:** Receives raw text data from users or from descriptive sentences (`‘comments‘`) in the dataset.
- **TextVectorization Layer:** Performs text normalization and converts words into integers (`tokens`). This layer is configured with a fixed string length to ensure consistency for the subsequent deep learning layers.
- **Embedding Layer:** Converts integer tokens into feature vectors in a **300-dimensional** space. The team uses `FastText` weights that have been pre-trained on the massive Vietnamese data set.
- **Multi-branch Conv1D Layers:** Data is passed through **3 parallel convolutional branches** with kernel sizes of **3, 4,** and **5**, respectively.
- **Multi-branch GlobalMaxPooling1D Layers:** After each Conv1D layer, Global Max Pooling extracts the largest value from each filter to identify the most important features in the input sentence.
- Concatenate Layer: Combines all prominent features from the three branches into a single vector, creating a comprehensive semantic and contextual representation of the query.

- **BatchNormalization Layer:** Normalizes the outputs from the concatenate layer, stabilizing the training process, minimizing reliance on initial weights, and accelerating convergence.
- **Dropout Layer:** Applies a dropout rate of `rate=0.5` during training to combat overfitting.
- **Dense Layer (hidden):** A layer fully connected to **256 neurons** and the `ReLU` activation function, enabling the model to learn more complex nonlinear feature combinations from text data.
- **ense Layer (output):** Uses the `Softmax` trigger function to convert scores into a probability distribution, thereby identifying the locations that best match the user's requirements.

5.5.2 Suitability for problem/data

The choice of **TextCNN** for this project is logical because:

- The input data mconsists of short/medium length review comments, which often contain phrases expressing very clear meanings or characteristics.
- TextCNN performs well in extracting **local features/n-gram** phrases through sliding windows of sizes 3, 4, and 5. This is perfectly suited to short, information-dense texts like comments.
- The problem focuses on the efficiency of the top choices rather than simply evaluating the most accurate result. Therefore, using the **Top-5 Accuracy** criterion perfectly reflects the nature of a Recommendation System.
- Compared to deep context models like RNNs, LSTMs, or large models like Transformers (BERT), TextCNNs possess extremely powerful parallel computing capabilities due to the nature of convolution. This accelerates convergence during training and achieves real-time prediction (inference).

5.5.3 Extracting comment points and comments

From the original comment data column, a list of text sentences (`comment_list`) is extracted and decomposed, playing a crucial role in the vectorization process and training of the deep learning model in the subsequent steps.

5.5.4 Creating comment for locations lacking data

During the analysis and survey of the dataset, the team noticed that some locations had absolutely no comment data from the data collection process. Therefore, the team implemented a method of aggregating **comment (category + address + price)** and assigned a default positive rating of **5.0**.

5.5.5 Restructuring the dataset and creating ranking features

- The original data structure was "**1 location - many comments**". To enable the model to learn each context independently, the team used the 'explode' function to decompose it into "**1 location - 1 comment**" row by row.
- In a recommendation system, having an accurate scoring system to rank locations in the output is crucial. The team created a new feature called `rank_score` through the following priority fill in strategy:
  - Prioritize using the `score` rating from the dataset.
  - For locations with missing scores, calculate the average of all existing comments at that location (`comment_score_mean`) to replace them.
  - If a location still has neither a rating nor a comment, take the average `rank_score` across the entire dataset.

5.5.6 Label encoding

The names of the locations are encoded into integer labels; this `label_id` variable acts as the **ground truth** (target variable) so that the model can understand and calculate the loss function during training.

5.5.7 Data splitting

Due to the nature of the real-world data, there is an imbalance phenomenon, with some locations having very few comments (less than 6 comments). If the data were randomly split, these samples might fall into the Validation' or 'Test' sets, causing the model to lose data for learning or resulting

in data splitting errors. To address these issues, the team implemented the following data splitting strategy:

- Separate locations with **fewer than 6 comments** (`rare_classes`).
- After the rare data is removed, instead of simply splitting the remaining data, the team applied **stratified sampling** (`stratify=y`). This means that each split is based on a ratio of **70% Training - 15% Validation - 15% Test**, while still ensuring an even number of samples across all three sets for each location.

5.5.8 Setting up for the machine learning process

- The TextVectorization Layer converts text strings into arrays of integers with a vocabulary space limited to the maximum of **20,000 tokens** (`max_tokens = 20,000`), and the output sentence length is fixed at **100 tokens**.
- The Embedding Layer receives and is responsible for representing the TextVectorization Layer's indices as contextually meaningful continuous vectors. Instead of initializing a random semantic model from scratch, the load group uses **a pre-trained FastText model** (Vietnamese version cc.vi.300.bin):
  - A carefully initialized embedding matrix (`embedding_matrix`) is created by traversing the vectorized dictionary above. This matrix extracts and stores the corresponding FastText semantic vector for each character. This matrix will then be loaded directly into the **Embedding layer** of the TextCNN network to provide rich language understanding right from the start.
- Integer classification labels (`y`) will be encoded one-hot into multidimensional binary vectors, which is the input standard for compatibility with the `CategoricalCrossentropy` loss function I used below.
- Loading the entire dataset and memory (RAM) at once would be inefficient; the team splits the data into `batch_size = 64` to balance convergence speed and hardware requirements.

5.5.9 Model training

To solve the recommendation problem from a multi-class classification perspective, the team built and compiled a TextCNN model with refinements to optimize feature extraction capabilities and minimize overfitting:

- **Embedding Layer** uses a frozen FastTest weight matrix (`trainable=False`), allowing the model to preserve strong contextual representation from the pre-trained model and significantly reduce computational costs.
- **Conv1D & GlobalMaxPooling1D** use kernel sizes of **3, 4, 5** and **128 filters** per branch, enabling the model to scan and extract N-gram semantic phrases of varying lengths.
- **Dropout** (`rate=0.5`) is set to randomly disable 50% of neurons during training to prevent rote learning. Finally, the signal passes through the hidden layer **Dense(256, ReLU)** before exiting the **Output(softmax)** layer.
- Using `Adam`, an algorithm that calculates and updates the neural network weights after each iteration to minimize error. In natural language processing, the gradient of rare words is often very small. Adaptive learning mechanisms help the model capture signals from these rare words extremely efficiently, resulting in very fast convergence and saving training time and resources.
- Using `CategoricalCrossentropy` combined with **label smoothing** (`label_smoothing=0.1`) for the loss function, in a problem requiring classification of thousands of different subclasses, reducing the absolute confidence in a single label helps the model avoid excessive penalties, thereby increasing generalization.
- Instead of absolute accuracy, the team evaluated the model using **Top-5 Accuracy**, a standard metric for recommendation problems.
- To control the training process over **30 epochs**, the team employs a monitoring strategy using a list of callbacks:
  - `EarlyStopping` by monitoring the loss function on the validation set (`val_loss`). If after 3 epochs the model shows no improvement, the training process stops immediately and the model restores its weights to the best point.

- When the model shows signs of plateauing on `val_loss` within **2 epochs**, the learning rate is halved (initially initialized to 0.001 due to the use of Adam).
- Due to data imbalances in the comments, the team adds a callback object to calculate and monitor the **F1-Score macro** (viewing each location independently) on the validation set at the end of each epoch.

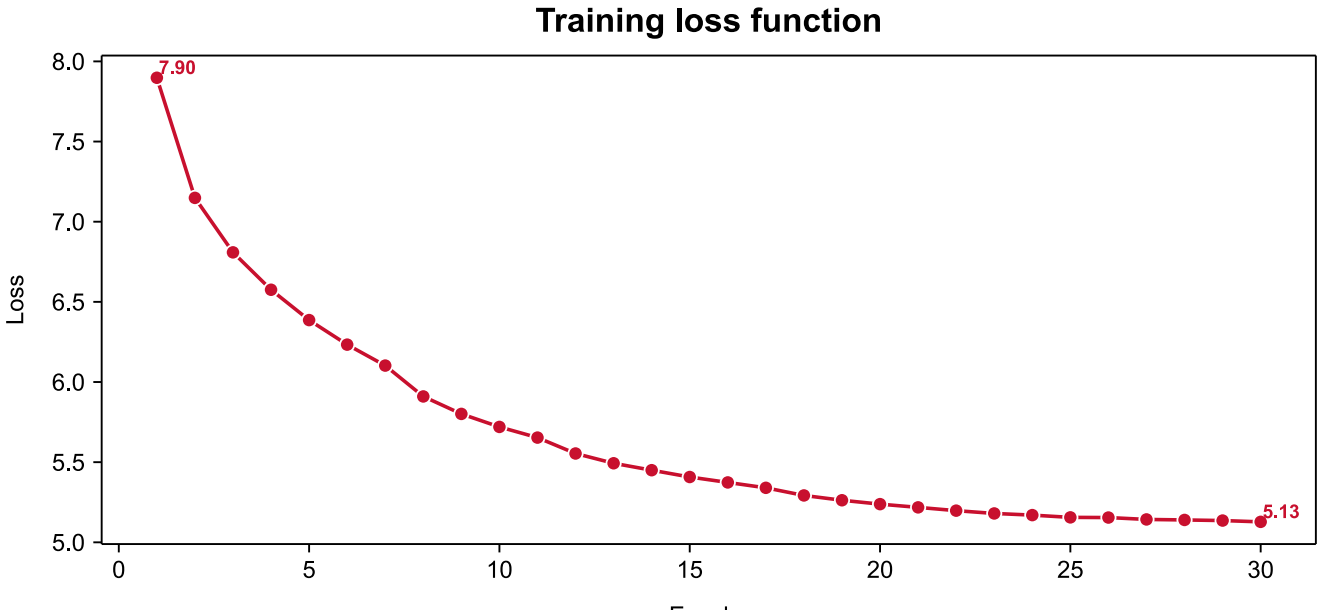

Figure 20: Training model on Train set

5.5.10 Data evaluation

After the training process is complete, the model will perform an independent evaluation on the **Test set** (`test_ds`). To evaluate the model, the team uses the `predict` function to output the predicted probability distribution for the entire test set and conducts analysis through measures such as: **Top-5 Accuracy**, **Macro F1-Score**, and a detailed classification report.

5.5.11 Retraining on full dataset

After training on the Train set and testing on the Test set, to ensure the model achieves the highest efficiency and accuracy before implementation in the recommendation system, the team will perform a retraining process to integrate all existing data.

- The classes will retain the same architecture as the training process above; however, the data used here will be **aggregated data** from all three sets: Train, Valid, and Test.
- In the data training for the Train set, the team used label representation using **One-hot encoding**. However, when loading tens of thousands of classification rows at once, the One-hot matrix would consume a huge amount of unnecessarily RAM. Therefore, the team decided to replace it with a technique that retains **integer label** and switch to using the corresponding functions/measures `SparseCategoricalCrossentropy` and `SparseTopKCategoricalAccuracy`.
- Since there is no longer a validation set to monitor and trigger the `EarlyStopping` mechanism, the team will extract the optimal loop (`best_epoch`) - the epoch that yields the lowest `val_loss` ever recorded in the previous training process. This number will be used as the absolute stopping condition for the model when training on the entire dataset.

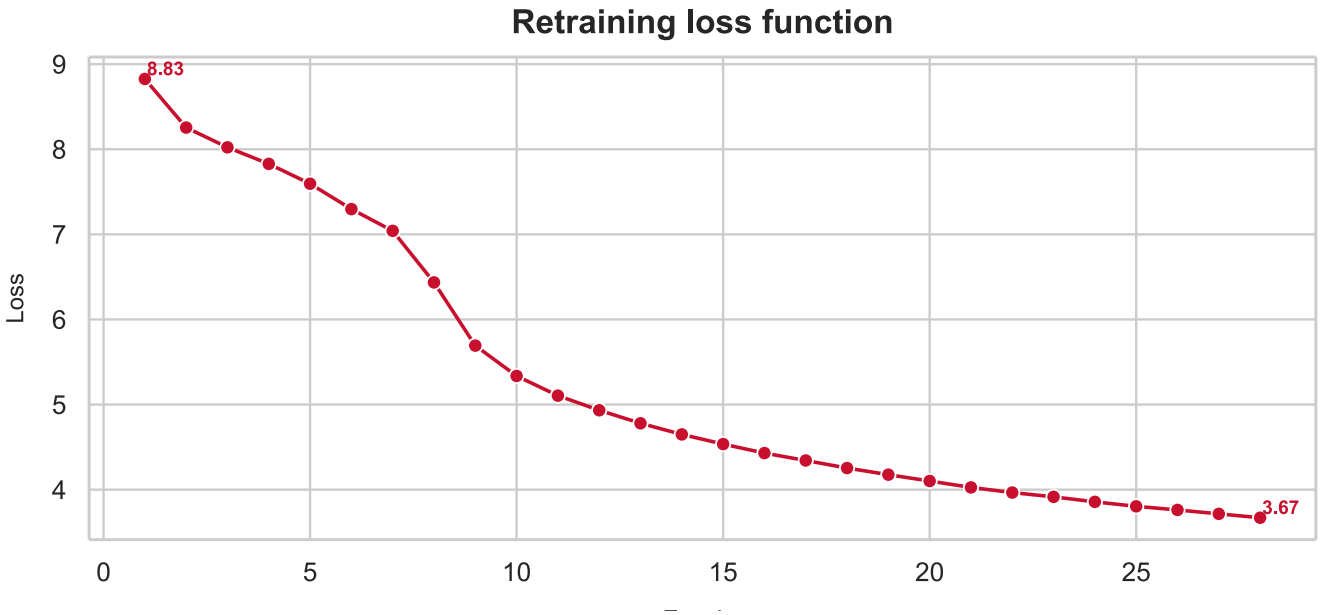

Figure 21: Retraining on full dataset

5.5.12 Save checkpoints

After completing the training process on the entire dataset, archiving is essential to allow the recommendation system to operate independently in different environments **without having to repeat the preprocessing or model training steps from scratch**.

This archiving is saved at "`./checkpoints/textcnn.pkl`", a dictionary named checkpoint built to centrally package all important parameters and data structures, including: model weights, vocabulary vectors, labels, and hyperparameters.

5.6 Traditional: Random Forest

The system uses **Traditional** paradigm as the main model for the problem, applying content-based recommendation and restating the problem in the form of multi-class classification by extracting features from user comments to predict the most suitable location.

5.6.1 Model architecture

The core model chosen is the **Random Forest Classifier**. Random Forest is a machine learning algorithm belonging to the **ensemble learning** group. Instead of using a single model, Random Forest builds multiple decision trees and combines their results.

- Input features: A combination of **text** and **numerical** data.
- Data processing: Using `TfidfVectorizer` to convert text into feature vectors. The model is limited by `max_features` and `ngram_range`. Normalize numeric features such as `score`, `nums_comments`, `price_min`, and `price_max`.
- Feature stacking: Concatenate the **TF-IDF** sparse matrix and the numeric matrix to form an input vector.
- Output: Convert to a probability distribution to determine the top 5 most suitable locations.

5.6.2 Suitability for problem/data

The choice of **Random Forest** for this interleaving method is entirely logical because:

- The data includes both **text** and **numeric** data. Random Forest handles both types of data well without requiring deep normalization like neural networks; only features need to be appropriately vectorized.
- A unique feature of Random Forest is that it is an **Ensemble** model consisting of multiple decision trees. These trees are trained independently, maximizing hardware utilization during

training, resolving data imbalances through random selection, and possessing self-evaluation capabilities.

5.6.3 Extracting comments

As detailed in the section 5.5.3 (page 37)

5.6.4 Creating comment for locations lacking data

As detailed in the section 5.5.4 (page 38)

5.6.5 Restructuring the dataset

As detailed in the section 5.5.5 (page 38)

5.6.6 Data splitting

As detailed in the section 5.5.7 (page 38)

5.6.7 Text normalization

Convert all comments to lowercase, removing special characters but retaining Vietnamese with diacritics. This helps the model focus on the actual content of the comment without being affected by formatting.

5.6.8 Vectorizing text

The team used **TfidfVectorizer** algorithm to transform text records and user queries into sparse matrix arithmetic vectors. The result is a matrix of size (**N_train, 5000**) where each row represents the ID of a location – the corresponding point of that location with the 5000 most frequently occurring and meaningful words in the entire dataset.

- **TF (Term frequency)** calculates the frequency of keyword occurrence in the document.
- **IDF (Inverse document frequency)** assigns high weight to rare/specific keywords and reduces the weight of meaningless words that appear too frequently.

- The `sublinear_tf=True` parameter is applied to avoid excessive keyword repetition that could skew the results.

5.6.9 Numerical features

Apply **StandardScaler** and fill in the missing values for the 4 numerical features (`score`, `nums_commnents`, `price_min`, `price_max`).

5.6.10 Merge features

Combine the sparse matrix **TF-IDF** with **numerical matrix** into a single matrix

5.6.11 Label encoding

Use **LabelEncoder** to convert each place name into an integer from **0** to **n-1**.

5.6.12 Model training

The model training process was designed with phased optimization to find the best set of hyperparameters and calculate the model's performance:

- Instead of **optimizing all parameters** simultaneously, which is very time-consuming, the team fixed the number of trees at a moderate level (`n_estimators = 50`) to focus on finding hyperparameters that shape the structure of the decision tree.
  - Tested various parameter combinations including the maximum tree depth (`max_depth`), the minimum number of samples to split a node (`min_samples_split`), and the minimum number of samples at a leaf node (`min_samples_leaf`).
  - For each combination, the model was trained on the Train set and its predictions were evaluated on the 'Validation' set. The model will maximize its performance based on the **Top-5 Accuracy** and **Macro F1-score** to determine the **best set of parameters** (`best_params`).
- Optimize the number of trees (`n_estimators`) using **Warm start** and the **Out of Bag** index.

- The `warm_start=True` parameter is used so that the model doesn't have to retrain from scratch. The model will retain the number of trees learned from the previous iteration and gradually increase the number of trees until the **best number of trees** (`best_n`) is found.
- Evaluation is done using OOB instead of Validation to avoid overfitting. Based on the Out of Bag index (`oob_score=True`), the team will use two unified metrics: Top 5 Accuracy and Macro F1 for monitoring.
- Implement an **EarlyStopping** mechanism while gradually increasing the number of trained trees. If the accuracy of Top 5 Accuracy does not increase compared to the previous step, the training process will stop to avoid overfitting and find the best number of trees (`best_n`).

  After optimizing the hyperparameters **best_params** and the number of trees **best_n**, the model will be trained on the 'Train' set and validated on the **Validation**' set using the **Top5 Accuracy** and **Macro F1** metrics.

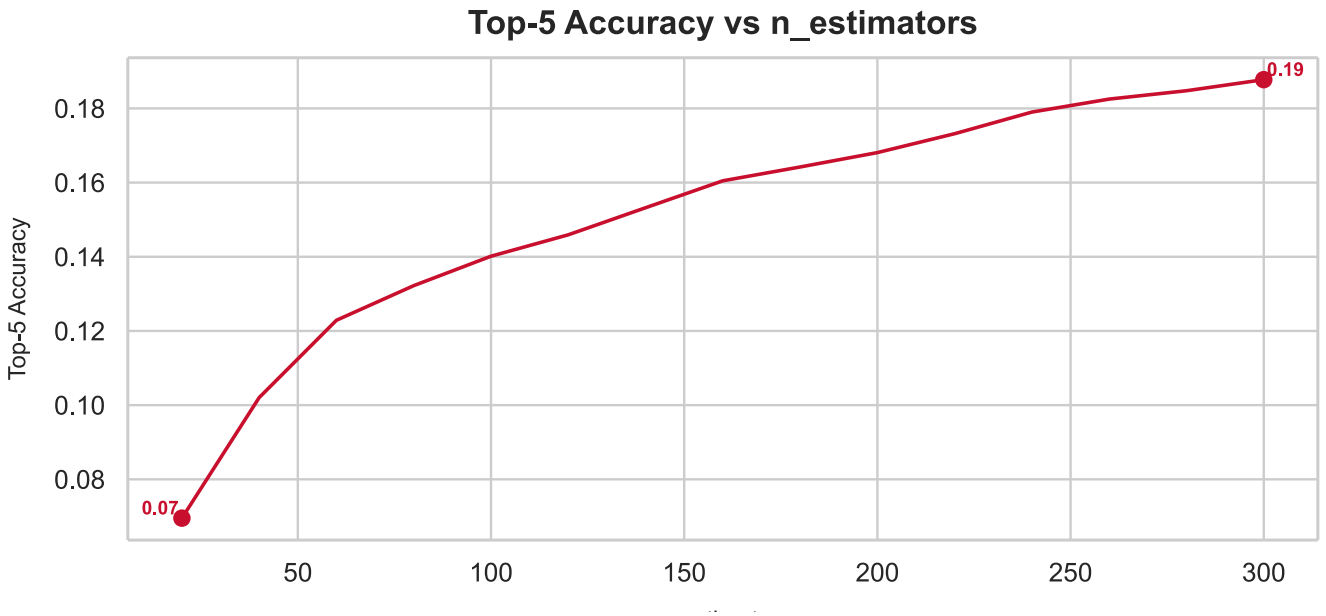

Figure 22: Training model on Train set

5.6.13 Model evaluation

The team will perform retraining on the **Train** and **Validation** datasets. The `Random Forest Regression` model is initialized with optimized hyperparameters and trained from scratch on this pooled dataset, allowing the model to maximize the use of available data to improve generalization

capabilities. Then, validation will be performed on the Test dataset using the Top5 Accuracy and Macro F1 measures.

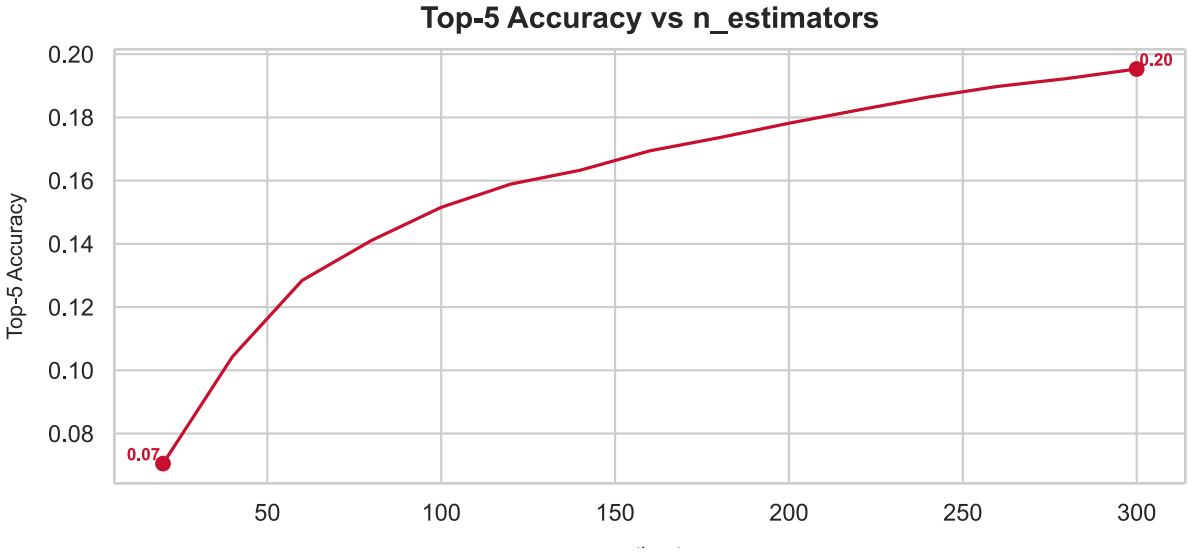

Figure 23: Training model on Train & Validation set

5.6.14 Retraining on full dataset

After training and testing the model on the Test set to evaluate its effectiveness and accuracy before implementing it into the recommendation system, the team will perform a retraining process to integrate all existing data (the entire process is carried out as in the previous stages).

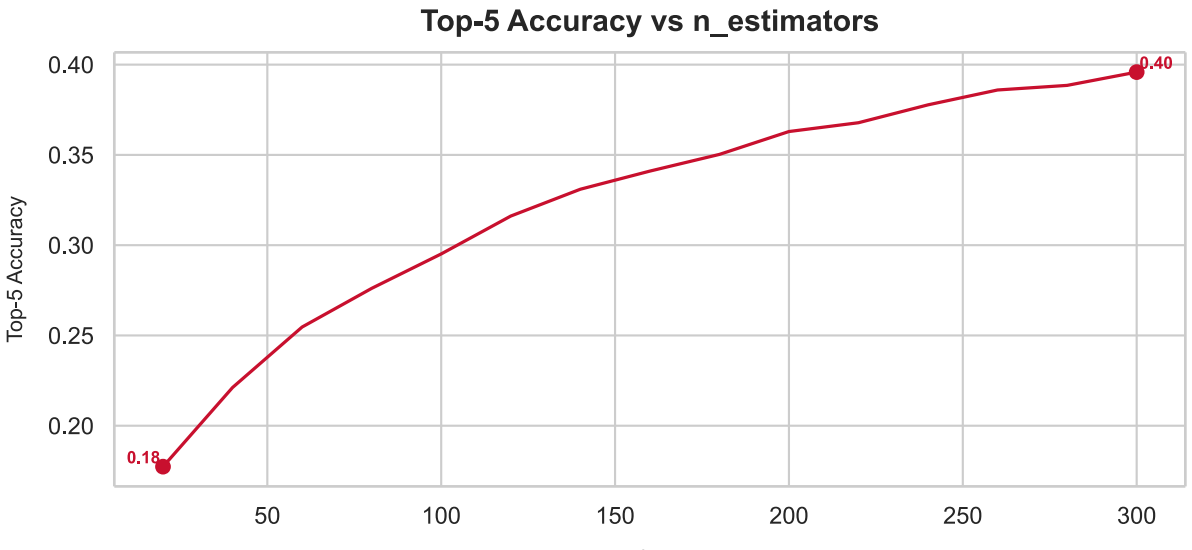

Figure 24: Retraining on full dataset

5.6.15 Save checkpoints

After completing training, all **artifacts** (model, vectorizer, scaler, etc.) need to be saved so they can be reloaded and used for Inference without retraining.

- `rf_model.pkl`: RandomForestClassifier fully trained with 200 trees
- `tfidf_vectorizer.pkl`: TfidfVectorizer with 5000 features trained on the training set
- `feature_scaler.pkl`: StandardScaler for 4 arithmetic features
- `label_encoder.pkl`: LabelEncoder for mapping location names to numerical codes
- `metadata_lookup.pkl**`: Dictionary {Name → Category, Address, Price_min, Price_max}

5.7 Hybrid: Content-based Filtering + XGBoost

The system uses **Hybrid** paradigm, combining Content-Based Filtering candidate extraction techniques and the XGBoost machine learning algorithm, to solve the multi-class classification problem.

5.7.1 Model architecture

The architecture is designed as a two-stage funnel, strictly adhering to the **standard retrieval & ranking** model:

- Content-based Filtering:
  - Role: Stage 1 - Candidate generation.
  - Justification: Given the problem's characteristics, which require narrowing down tens of thousands of locations in milliseconds, and dealing with core data type being text data, the team decided to use a **Content-based** model through the `TF-IDF` algorithm. Instead of using complex Deep Learning models (such as LSTM or Transformer-based) that cause high latency, TF-IDF handles sparse matrices extremely efficiently, helping to quickly narrow down the **top100** most promising candidates based on consine similarity.
- XGBoost (learning to rank):
  - Role: Stage 2 - Reranking.

– Justification: Linguistic similarity alone is insufficient to determine location quality. In the second funnel layer, the data has been transformed into tabular data containing numerical variables (price) and categories. The team chose the **XGBRanker** algorithm (a boosting model belonging to the Traditional machine learning group). This decision is justified by the superior power of the decision tree structure in processing tabular data, its good noise immunity without requiring overly strict normalization. In particular, this algorithm directly addresses the essence of the ranking problem through the **LambdaMART** variant, optimizing directly the `NDCG` loss function to re-rank 100 candidates as accurately as possible.

5.7.2 Suitability for problem/data

Combines two algorithms to maximize the use of existing data:

- Reasons for choosing **Content-based Filtering** (TF-IDF) is extremely fast and efficient in finding text similarities. It acts as a wide "funnel" scanning tens of thousands of locations and filtering out a list of potential candidates whose comments and descriptions match the user's keywords.
- Reasons for choosing **XGBoost** (XGBRanker) because TF-IDF only focuses on text, ignoring factors that are extremely important to travelers such as: price, rating score, and number of comments. `XGBRanker` belongs to the **Gradient Boosting** algorithm family, capable of combining text similarity scores with tabular data to directly optimize rankings (maximize the `NDCG` index), resulting in the most accurate and realistic final results.

5.7.3 Extracting comments

As detailed in the section 5.5.3 (page 37)

5.7.4 Creating comment for locations lacking data

As detailed in the section 5.5.4 (page 38)

5.7.5 Restructuring the dataset

As detailed in the section 5.5.5 (page 38)

5.7.6 Data splitting

As detailed in the section 5.5.7 (page 38)

5.7.7 Semantic item profiling

To enable the system to understand the characteristics of each location, the team compiled a comprehensive textual profile. Interpolation techniques were applied by combining **structured metadata (categories, addresses)** with **unstructured data (all comments)** into a single document. This method overcomes information fragmentation, creating a unified semantic space that fully reflects both the physical characteristics and the real-world customer experience.

5.7.8 Vectorizing text

As detailed in the section 5.6.8 (page 44)

5.7.9 Cosine similarity

When a query is entered, the model also vectorizes this query and calculates the **cosine similarity** between the **query vector** and the **entire matrix of location vectors.** Locations with smaller cosine angles (closer to 1) have higher similarity.

5.7.10 Recall@K evaluation

Since this is only a rough filtering step, the most important evaluation metric here is not the absolute exact ranking, called **Recall@K** (coverage). The team calculates the percentage of the selected **Top 100** candidates that contain the exact location the user is looking for in Ground Truth. If the `Recall` is high, **stage 2** (`XGBoost`) will have a quality input data source to refine the ranking.

5.7.11 Ranking learning dataset

XGBoost requires tabular input data for its specific structure. The `LearningToRankDataset` class will transform the raw **Top 100** candidates into a standardized dataset (feature matrix `X(N_train * 100, features)`, label `y(N_train * 100)`, and cluster array `qid(N_train * 100)`) through streamlined processing steps:

- Labeling initialization:
  - **Label 1:** The candidate is the actual location the user visited (ground truth).
  - **Label 0:** The remaining candidates in the `Top 100` list.
- Feature engineering
  - Add metadata (`nums_comments, score`).
  - Logarithmic transform (`np.log1p`): Applied to `price` to narrow variance, helping the learning model become stable and not be disturbed by locations with prices in the tens of millions of VND.
  - One-hot encoding: Converts `Category` into binary columns so that XGBoost understands the specific characteristics of location types such as `Attraction, Dining`, or `Hotel`.
- Query grouping: All data must be grouped and sorted by `query_id` so that the algorithm knows which candidates are directly competing with each other in the same search.

5.7.12 The XGBRanker algorithm and the NDCG objective function

Underlying, **XGBoost** applies a variant of the `LambdaMART` algorithm. Instead of predicting an absolute score, it directly optimizes the relative ranking of the list. The `NDCG` mechanism ensures two principles:

- Relevance: Candidates with good characteristics (text match, high rating, reasonable price) must be given a high score.
- Position discount: Correct results that are pushed down to position 10 will be heavily penalized (the discounted value divided by the logarithm of the position) compared to being ranked at the top 1.

5.7.13 Model training

The system takes the Content-based filtering model from **phase 1** as input to extract the **top 100** candidates, then proceeds to build the `XGBRanker` model.

- It performs an **optimal** iterative search (`best_iteration`) using an `EarlyStopping` mechanism on the Validation set to prevent overfitting.
- The iterative loop that yields the highest score (`ndcg@5`) on the Validation set is saved as the `best_iteration`. This is the perfect balance point, making the model intelligent enough to predict accurately without being too complex to be noisy. The `best_iteration` will be extracted for retraining on the entire dataset in the next step.

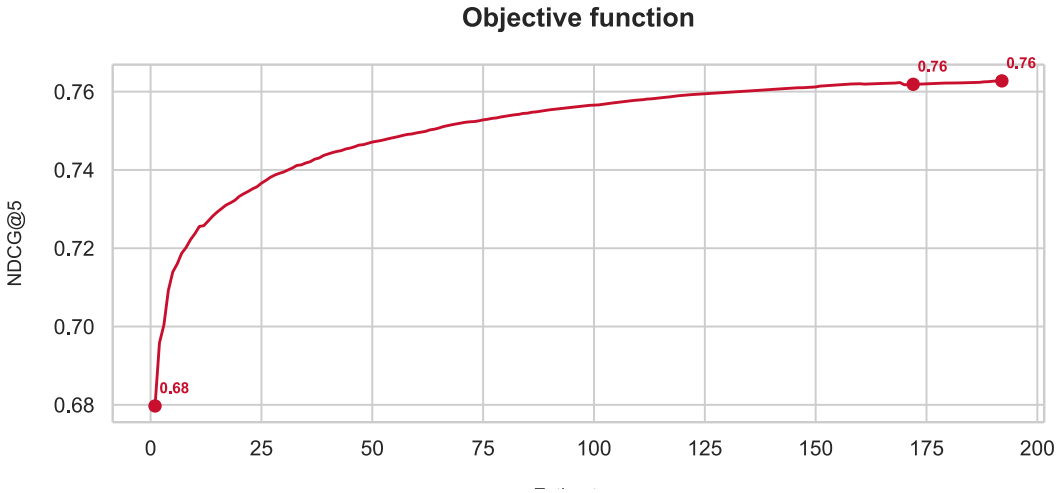

Figure 25: Training model on Train set

5.7.14 Model evaluation

Due to the implemented **EarlyStopping** mechanism, the model has found the optimal number of iterations (`best_iteration`).

The team will retrain on the **Train** and **Validation** sets. The **XGBoost** model is reinitialized and trained from scratch on this pooled dataset with the exact optimal number of iterations found, allowing the model to maximize the use of available data to improve its generalization capabilities. Then, validation will be performed on the Test dataset using the Top5 Accuracy and Macro F1 measures.

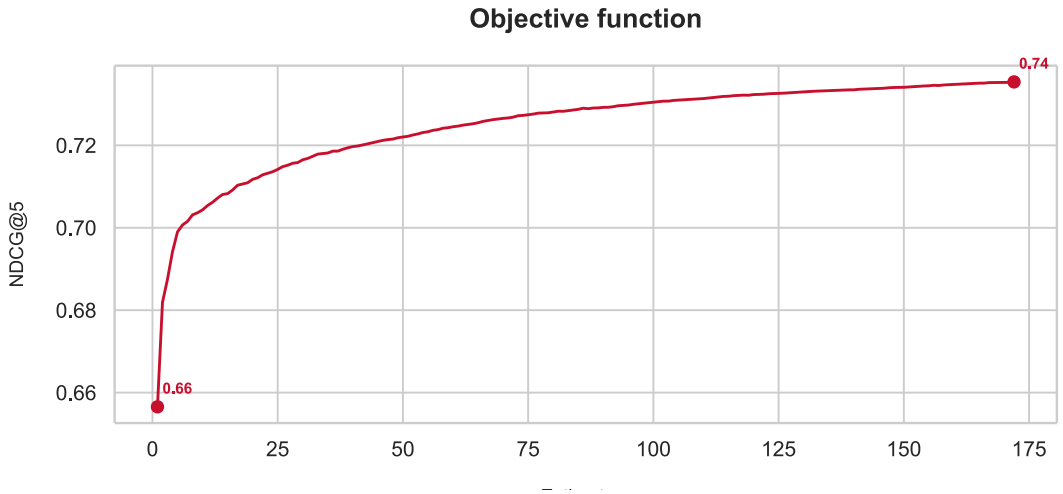

Figure 26: Training model on Train & Validation set

5.7.15 Retraining on full dataset

After training and testing the model on the Test set to evaluate its effectiveness and accuracy before implementing it into the recommendation system, the team will perform a retraining process to integrate all existing data (the entire process is carried out as in the previous stages).

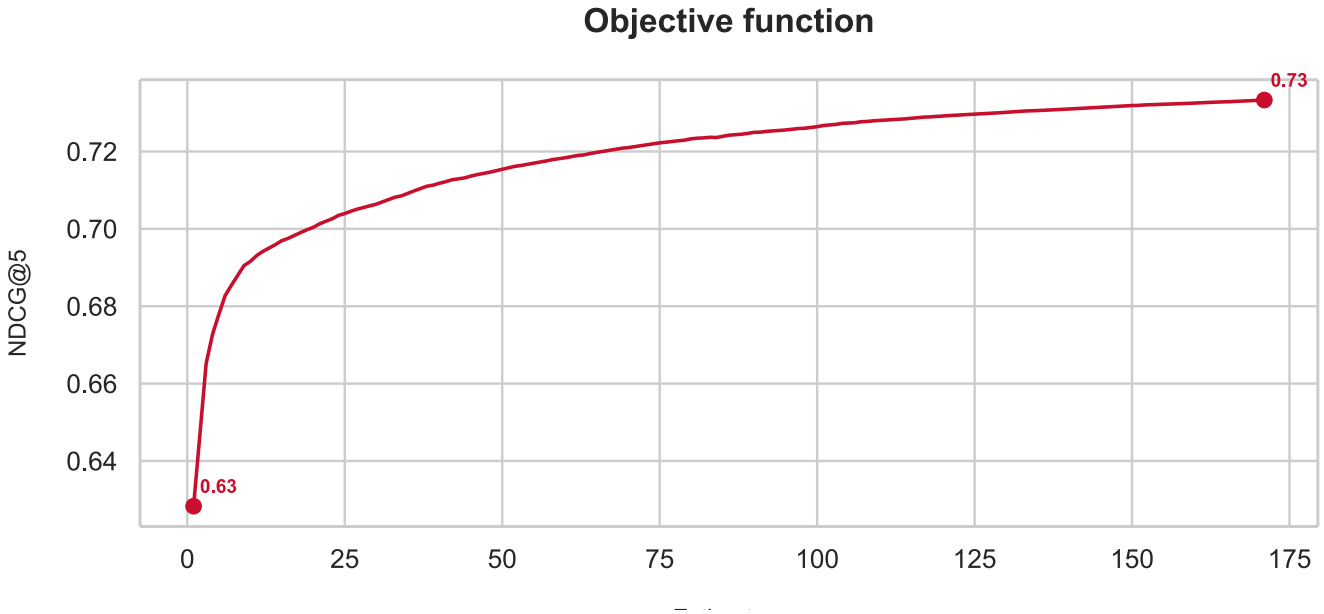

Figure 27: Retraining on full dataset

5.7.16 Save checkpoints

After completing the training process on the entire dataset, archiving is necessary so that the system can operate independently without needing to be retrained from scratch.The archive is located at "`./checkpoints/`", including 4 `.pkl` files:

- CBF: TF-IDF filter and location matrix
- Encode: One-Hot encoder for Category
- XGBRanker: Ranking learning model
- Metadata: System configuration information

6 Analysis and discussion

6.1 Performance comparsion

Model	Top-5 accuracy	Macro-F1 score
`TextCNN`	19.64%	0.06
`Random Forest`	26.42%	0.06
`CBF + XGBoost`	27.09%	0.09

Table 4: Performance comparison

Summary

- For the **hybrid paradigm** (CBF + XGBoost): The use of a two stage funnel architecture has proven remarkably effective. **Stage 1** (Content-based Filtering with `TF-IDF`) helps the system quickly narrow down the search area. **Stage 2** (`XGBRanker`) not only learns text similarities but also thoroughly exploits key numerical features crucial to the travel experience (such as price, score, `nums_comments`). Instead of predicting classification labels, directly optimizing rankings through the objective function `rank:ndcg` pushes the most relevant locations to the top.
- For the **traditional paradigm** (Random Forest): Performance will be lower because of the significant barrier to solving the problem from a multi-class classification perspective across a huge label space.
- For the **deep learning paradigm** (TextCNN): The performance difference between TextCNN architecture and traditional/hybrid methods lies in their **text vectorization strategies**. Although both process natural language input, their approaches are completely different. One limitation is the **semantic similarity in TextCNN**; in location suggestion problems (comment - location name), focusing on semantics leads to semantic dilution.

6.2 Complexity and storage cost

Model	Storage cost	Parameters	Training time	Inference time
`TextCNN`	32.29MB	8.099.939	2090.53s	0.06s
`Random Forest`	2695.13MB	8.987.684.400	1085.26s	0.15s
`CBF + XGBoost`	35.75MB	32.388	261.57s	0.08s

Table 5: Complexity and storage cost

Summary

- For the **traditional paradigm** (Random Forest): Storing all tree configurations creates a huge number of parameters, and loading the entire model would be resource-intensive for machines with limited resources.
- For the **hybrid/deep learning paradigm** (CBF + XGBoost, TextCNN): Both perform well in compressing learning information, making storage and deployment easier.

6.3 Discussion

The shift from a **recommendation system** to a **multi classification machine learning** problem, which results in lower model performance. The nature of this project is **the application of machine learning models** to enhance Vietnam's tourism industry. To implement and evaluate these models, transforming the problem into a multi-class classification approach is mandatory. However, the team will perform post-processing and inference based on the model results (the final application output) to optimize the generation of a suitable list of suggested locations from the user's input description.

- Recommendation require a diverse and relevant `topK` classification, while multi-class classification optimizes for a single class. When a query has multiple relevant locations, the classification problem forces the selection of only one ground truth, resulting in a lower score even if the actual results are still valid.
- The label in the data is "**locations users have visited**" not **"best location"**. There may be many equivalent locations, but the dataset only assigns one label, leading to a penalty for the model making other valid choices.

- Comments are short, general, and lack distinctive features that differentiate locations of the same type (many attractions/dinings/hotels have similar descriptions). This limits the ability to differentiate classes.
- There is an imbalance between classes in the data; some locations have many comments while others only have one or two.

7 Application development

The integrated **Content-Based Filtering** and **XGBoost** rating model demonstrated superior performance in the previous experiments. To bring this model to a practical user-server application, the team discovered the need to develop a web application called **Vietnam Travel Planner** — a travel advisory system for Ho Chi Minh City.

7.1 System architecture

The application is designed with a completely separate **client-server** architecture, separating the frontend and backend, and communicating via RESTful APIs

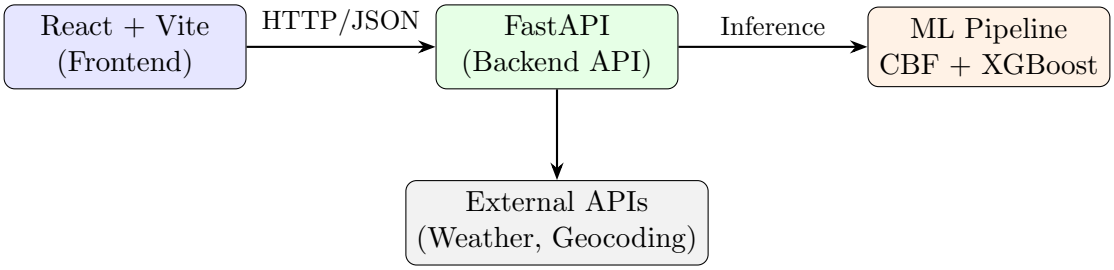

Figure 28: System architecture of Vietnam Travel Planner.

7.1.1 Backend (Python - FastAPI)

The backend is built using the **FastAPI** framework with a clear modular structure:

- **Routers** defines API endpoints (`/api/recommend, /api/weather, /health`).
- **Services** contains the main business logic — the `recommender.py` module integrates a two stage inference pipeline; the `weather.py` module connects to the **OpenWeatherMap API**; the `geocoding.py` module looks up location coordinates.
- **Models** ensures rigorous input/output validation.
- **Core** manages configuration (environment variables) and dependency injection.

Machine Learning models (`TF-IDF vectorizer, XGBoost Ranker, encoder`) are serialized using `joblib` as **.pkl files** and loaded into memory when the application starts via **FastAPI's lifespan** mechanism, ensuring fast response times for each request.

7.1.2 Frontend (React - Vite)

The user interface uses React in combination with the **Vite** build tool, **Tailwind CSS** for styling, and **Framer Motion** for motion effects.

7.1.3 Deployment

To bring the application into a real world environment and ensure high availability, the system has been deployed on modern **cloud platforms**:

- **Backend** (FastAPI & Machine Learning models) deployed on a **rendering** platform. The system automates the build process and provides stable API endpoints to handle inference requests from users.
- **Frontend** (React + Vite) deployed on **Vercel**, a platform providing a global content delivery network (CDN), optimizing page load times and delivering a smooth and high-speed experience.

In addition, the project maintains **Docker** packaging with the `docker-compose.yml` file to support consistent development and testing on the local environment.

7.2 Inference pipeline

When a user enters a search query, the system performs a inference pipeline:

- **Step 1** (Candidate retrieval): The user's query is vectorized using TF-IDF and matched cosine similarity with over 500 item profiles. The system extracts the **top100** potential candidates from the content-based filtering.
- **Step 2** (Hard filtering & fallback): Hard filtering is applied based on district/county and maximum budget. The system has a three-level fallback mechanism:
  - Loosen the budget by an additional **20%** if not enough results are found.
  - Completely remove the budget constraint if it is still insufficient.
  - Expand the search scope if the specified area does not have enough services.

- **Step 3** (XGBoost re-ranking): The **top100** candidates after filtering are injected into the `XGBoost Ranker` model to calculate relevance score, with input characteristics including: cosine similarity, log-price, score, number of comments, and one-hot category encoding.
- **Step 4** (Business logic): The system selects the best **1 hotel, 2 dinings**, and **3 attractions**. Locations in the same district as the hotel are prioritized (20% penalty for locations in different districts) to optimize travel routes.

7.3 User interface and main components

The application interface is designed on the principle of simplicity and intuitiveness, allowing users without technical knowledge to use it easily.

- **Smart search bar:** Users enter their requests in Vietnamese, for example: "Khách sạn yên tĩnh gần Quận 1" or "Quán ăn ngon, giá bình dân". The search bar integrates a typewriter animation effect displaying suggested examples, helping new users understand how to use it. In addition, quick suggestions allow for one-click searching.
- **Advanced filter panel:** The system provides two filtering criteria:
  - Supports selecting multiple districts simultaneously (16 districts/counties/cities in Ho Chi Minh City) via dropdown checkbox.
  - Supports custom input or quick selection from preset levels (100K, 500K, 1M, 2M, 5M VND).
- **Result card:** Each suggested location is displayed as a visual card with complete information:
  - Illustrative images categorized by type (hotels, dinings, attractions).
  - Color badges (blue – hotels, orange – dinings, purple – attractions).
  - Rating, address, operating hours, price range.
  - "Add to map" button to mark the location on the interactive map.
  - External link to the original details page.
- **Travel map:** The map uses the `Leaflet` library with tile layers from **OpenStreetMap**, limiting the display area to the **Ho Chi Minh City** area. Features include:

  - Customizable markers with colors to distinguish locations by category.
  - Information popup displaying name, address, and price when clicking on the marker.
  - Displays the route connecting the selected points using a dashed line with animation effects.
  - Autofit bounds automatically adjusts the zoom to display all markers on the screen.
  - Precise coordinates are retrieved from a geocoding database (`JSON` file) with fuzzy matching support (85% similarity threshold).

- **Real time weather:** The weather widget displays a floating pill on the map, providing current temperature, perceived temperature, and humidity information from the **OpenWeatherMap** API. The system automatically alerts if the weather is unfavorable for outdoor activities (rain, storm, snow).

- **Budget estimate:** The budget popup summarizes the estimated costs for selected locations, separated into three categories (hotels, dinings, attractions) with an pie chart using the `Recharts` library.

7.4 Error handling and stability

The application is designed with comprehensive error handling capabilities on both the frontend and backend:

- **Backend:** Pydantic validation is used to check input (query length 1–500 characters, value 0). HTTP exceptions are handled with appropriate error codes (**503** – Service unavailable when the model is not ready, **404**, **401** for external API errors). The timeout for weather requests is set to 10 seconds.

- **Frontend:** Loading status is displayed using skeleton animation. Error messages are clearly displayed on the interface when the request fails. Input is disabled during processing to prevent duplicate submissions. Images have a fallback when they fail to load.

- **Fallback mechanism:** As described in the inference pipeline, the system automatically loosens the search criteria instead of returning empty results, along with an explanatory message for the user.

7.5 Conslusion

The **Vietnam Travel Planner** application has demonstrated the feasibility of deploying the Machine Learning model into a real world product serving end users. The system meets the criteria for user-friendliness (intuitive interface, natural language support for Vietnamese), full functionality (data entry, AI inference, visualization of results on maps and charts), stability (error handling, intelligent fallback mechanism), and responsiveness (model pre-loaded into memory, asynchronous API). Successful deployment on **cloud platforms** such as Render and Vercel also confirms the practicality and scalability of the system in the future.

8 Conclusions and future work

8.1 Conclusions

- In this lab, the team developed a **tourist destination recommendation system for Ho Chi Minh City** based on a multi-class classification problem, combining text data with numerical features. The source code was implemented in three main approaches: **traditional paradigm** (Random Forest), **hybrid paradigm** (Content-based Filtering + XGBoost Ranker), and **deep learning paradigm** (TextCNN).

- Systematically, the pipeline fully supports all stages from preprocessing, features engineering, training, evaluation, checkpoint storage to inference and post-processing to return a list of recommendations as requested **(3 Attractions, 2 Dining, 1 Hotel)**. This ensures the model not only achieves academic merit (measured by **Top-5 Accuracy**, **Macro F1**) but is also usable for practical implementation.

- The **hybrid paradigm** has the bes demonstrate of ranking capabilities by utilizing Content-based Filtering to narrow the space and XGBoost to optimize ranking. As a result, the recommendation system is of reasonable quality, suitable for demo use, and has the potential for expansion.

8.2 Future work

- Data expansion & automation:
  - Develop an automated data pipeline that runs periodically to collect new comments, update prices, and operating hours continuously. This will prevent the model from becoming outdated over time.
  - Expand the system's geographical reach to other major tourist centers in Vietnam (Hanoi, Da Nang, etc.).
- Personalize the suggested list based on user descriptions (location, interests, budget, etc.)
- Add advanced features to the app such as timetable looping, displaying optimal routes, etc.

9 Relevant links

Application repo: https://github.com/tphat2205/HCM-Trip-Planner

Application link: https://hcm-trip-planner.vercel.app/

Machine learning checkpoints: https://drive.google.com/drive/folders/1l0caQ4srtrgOgc4sOkse8I7Zb2pMDFny

Video demo: https://drive.google.com/drive/folders/1l0caQ4srtrgOgc4sOkse8I7Zb2pMDFny

10 Acknowledgment

To successfully complete **Lab 01: Machine Learning Applications for Promoting Vietnam Tourism**, We have received invaluable support, guidance, and encouragement from various individuals and sources. This project, undertaken as part of **CSC14005 - Introduction to Machine Learning**, has been a significant learning experience.

First and foremost, we extend our deepest gratitude to **Mr. Bùi Tến Lên, Mr. Lê Nhựt Nam** and **Mr. Võ Nhật Tân** - our instructors. Their insightful guidance, continuous support, and patient instruction throughout every stage of this project were instrumental in shaping our understanding of Data Science concepts and bringing this report to fruition. His expertise truly illuminated the complexities of problem-solving.

Our work also greatly benefited from consulting various academic resources, including research papers, specialized books, and reports from numerous authors and institutions. Despite our best efforts and dedication, We acknowledge that this report may contain certain limitations or areas for further improvement. We sincerely hope that **Mr. Lê Nhựt Nam, Mr. Võ Nhật Tân**, and anyone interested in this topic, will continue to provide us with valuable feedback and suggestions.

Specifically, we utilized **Gemini 3.1 Pro** and **GitHub Copilot** as valuable assistants for a **wide range of tasks**. Their support were significant in the **technical implementation**, from **suggesting methods to optimize our source code** and **using some effective tools** such as **using API for preprocessing, optimized by Adam, OOB and deployed by Docker**. Furthermore, they provided instrumental support on the documentation front **structuring the report in LaTeX, translating to English, ...**

Once again, we extend our sincere thanks to all who contributed to this project.

Group 3

15th April, 2026

References

1. FastText (2024). Word vectors for 157 languages. https://fasttext.cc/docs/en/crawl-vectors.html.

2. Yoon Kim (2014). Convolutional Neural Networks for Sentence Classification. https://arxiv.org/abs/1408.5882.